

Positive Lower Density of Collatz Predecessors

A detailed exposition with formal verification

12 September 2026

Abstract

We give a detailed exposition of the positive lower density of the predecessors of a fixed Collatz target. A positive target has a predecessor set of positive lower natural density if and only if it is not divisible by three. The proof uses inverse arithmetic, nonperiodic orbit points, and the analytic inverse-path estimates of Mazur. We state the analytic inputs separately and derive the counting conclusion step by step, including the multiplicity bound and the passage from residue averages to counts of distinct integers. We also prove the resulting criterion for universal convergence. The positive-density result and its counterexample consequence occur in Mazur's companion paper; they are not asserted here as new discoveries. A verification appendix records the corresponding Lean development.

Contents

1	Statement and notation	2
2	Inverse arithmetic in every ternary residue class	2
3	Nonperiodicity and unique hitting times	3
4	Finite measures and a precise analytic input	4
5	Choosing one seed with a positive limiting mark	6
6	From path weights to distinct sources	6
7	Removing the reference-density mark	7
8	A lower bound at every sufficiently large cutoff	8
9	All positive targets and the divisible-by-three exception	8
10	The exact consequence for possible counterexamples	8
A	Verification and reproducibility	10
B	Lean source: complete local dependency chain	11

1 Statement and notation

Let $\mathbb{N} = \{0, 1, 2, \dots\}$ and $\mathbb{N}_{>0} = \mathbb{N} \setminus \{0\}$. The ordinary Collatz map is

$$T(n) = \begin{cases} n/2, & 2 \mid n, \\ 3n+1, & 2 \nmid n. \end{cases} \quad (1)$$

We write $n \longrightarrow a$ if $T^k(n) = a$ for some $k \in \mathbb{N}$; the case $k = 0$ is allowed. For $a > 0$ and integer $X \geq 0$, set

$$R_a = \{n > 0 : n \longrightarrow a\}, \quad P_a(X) = \#\{n \in R_a : n < X\}. \quad (2)$$

Throughout, counts are of distinct positive integers and use a strict cutoff.

Theorem 1 (Target-density characterization). *For every $a \in \mathbb{N}_{>0}$,*

$$(\exists c_a > 0 \exists X_a \in \mathbb{N} \forall X \geq X_a : P_a(X) \geq c_a X) \iff 3 \nmid a. \quad (3)$$

The positive direction and the consequence for counterexamples are treated in [2]. The analytic construction used below is from the supporting development of [1]. We give the elementary arguments and the deductions from its analytic estimates in full. Those estimates are cited results: their Fourier and inverse-path construction is not reproved in this paper. This distinction fixes the scope of the proof and its dependence on earlier work.

For any set $E \subseteq \mathbb{N}_{>0}$ its lower natural density is

$$\underline{d}(E) = \liminf_{X \rightarrow \infty} \frac{\#(E \cap [0, X))}{X}.$$

An eventual bound by cX , with $c > 0$, implies $\underline{d}(E) \geq c$. Conversely, if $d = \underline{d}(E) > 0$, the definition of a lower limit gives $\#(E \cap [0, X))/X \geq d/2$ for every sufficiently large X . Thus (3) is precisely the assertion $\underline{d}(R_a) > 0 \iff 3 \nmid a$.

2 Inverse arithmetic in every ternary residue class

For positive m , let $v_p(m)$ denote the exponent of the prime p in m . The odd part is $\text{odd}(m) = m/2^{v_2(m)}$. On positive odd integers the Syracuse map is

$$S(n) = \text{odd}(3n+1). \quad (4)$$

If $v = v_2(3n+1)$, then $T^{v+1}(n) = S(n)$. Repeating this observation shows that Syracuse reachability implies ordinary Collatz reachability. We use $Q_a(X)$ for the number of positive odd $n < X$ with $S^k(n) = a$ for some $k \geq 0$.

Lemma 2 (An odd child). *Every positive odd a not divisible by three has an odd positive c such that $S(c) = a$.*

Proof. If $a \equiv 1 \pmod{3}$, put $c = (4a-1)/3$. Otherwise $a \equiv 2 \pmod{3}$, and put $c = (2a-1)/3$. In either case the numerator is positive, odd, and divisible by three; division by the odd number three preserves oddness. In the two cases respectively, $3c+1 = 4a$ and $3c+1 = 2a$. Since a is odd, removing exactly the displayed power of two leaves a . $\square$

Lemma 3 (Valuation calculation). *For $n > 0$, $v_3(4^n - 1) = 1 + v_3(n)$.*

Proof. Write $n = 3^t u$, where $3 \nmid u$. The factorization

$$4^u - 1 = 3(1 + 4 + \dots + 4^{u-1})$$

has second factor congruent to $u \not\equiv 0 \pmod{3}$, so its valuation is exactly one. If $z = 1 + 3h$, then

$$z^2 + z + 1 = 3(1 + 3h + 3h^2),$$

which has valuation one. Consequently, whenever $z \equiv 1 \pmod{3}$,

$$v_3(z^3 - 1) = v_3(z - 1) + 1.$$

Starting at $z = 4^u$ and cubing t times proves the formula. $\square$

Define $u_j = (4^j - 1)/3 = 1 + 4 + \dots + 4^{j-1}$, with $u_0 = 0$. For $j \geq 1$, u_j is odd, $u_j \geq j$, and $S(u_j) = 1$. Direct expansion gives

$$u_{m+n} = u_m + 4^m u_n. \quad (5)$$

For $n > 0$, Lemma 3 gives $v_3(u_n) = v_3(n)$; for $n = 0$ divisibility by 3^q holds on both sides. Because 4^m is a unit modulo 3^q , (5) yields

$$u_{m+n} \equiv u_m \pmod{3^q} \iff 3^q \mid n. \quad (6)$$

If $0 \leq i < j < 3^q$, their difference $j - i$ is not divisible by 3^q , so u_i, u_j have different residues. There are 3^q choices and 3^q residues; hence $j \mapsto u_j$ is a permutation modulo 3^q on this range. Replacing an index j by $j + t3^q$ preserves its residue and makes u_j arbitrarily large. In particular, every residue contains arbitrarily large odd one-step predecessors of one.

Lemma 4 (Affine transport). *Let c be as in Lemma 2. Every residue class modulo 3^q contains arbitrarily large odd M with $S(M) = a$.*

Proof. Set $A = 3c + 1$ and $F_c(x) = Ax + c$. Expanding both sides gives

$$3F_c(x) + 1 = 3Ax + 3c + 1 = A(3x + 1).$$

The identity $\text{odd}(rs) = \text{odd}(r)\text{odd}(s)$ follows by adding their powers of two. Thus if $S(x) = 1$,

$$S(F_c(x)) = \text{odd}(A)\text{odd}(3x + 1) = a \cdot 1 = a.$$

Since c is odd and A even, $F_c(x)$ is odd. It also exceeds x . Finally $A \equiv 1 \pmod{3}$, so A has an inverse modulo 3^q . For a prescribed residue r , choose $x = u_j$ in the residue $A^{-1}(r - c)$, with x larger than the prescribed size bound. The preceding permutation argument permits this choice. Then $M = F_c(x)$ has every required property. $\square$

3 Nonperiodicity and unique hitting times

A point x is *nonperiodic* for f if $f^k(x) \neq x$ for all $k > 0$. This definition does not exclude a point which later enters a cycle.

Lemma 5. *If $f(x) = f(y)$ and both points are periodic, then $x = y$. If a point is nonperiodic, each forward orbit visits it at most once.*

Proof. Choose positive periods p, q for x, y . Iteration gives $f^{pq}(x) = x$ and $f^{pq}(y) = y$. Applying f^{pq-1} to their common image proves $x = y$. For the second assertion, if $f^d(z) = f^e(z) = x$ with $d < e$, then $f^{e-d}(x) = f^{e-d}(f^d(z)) = f^e(z) = x$, contradicting nonperiodicity. $\square$

Theorem 6 (Nonperiodic seed supply). *For odd $a > 0$ with $3 \nmid a$, for every $q, B \in \mathbb{N}$ and every residue $r \pmod{3^q}$ there exists M such that*

$$M > B, \quad M \text{ odd}, \quad S(M) = a, \quad M \equiv r \pmod{3^q}, \quad M \text{ nonperiodic for } S. \quad (7)$$

Proof. Lemma 4 gives $x > B$ and then $y > x$ in the prescribed residue, with $S(x) = S(y) = a$. Since $x \neq y$, Lemma 5 prevents both from being periodic. Choose the nonperiodic one. $\square$

4 Finite measures and a precise analytic input

We now specify the analytic results used from [1]. This section separates the cited estimates from the arguments proved below; a supply of residues by itself is not a density theorem.

4.1 Reference densities

Let μ_0 be the point mass on $\mathbb{Z}/1\mathbb{Z}$. Inductively, sample Y with law μ_q and, independently, a positive integer V with $\Pr(V = v) = 2^{-v}$. Define μ_{q+1} as the law of

$$2^{-V}(3Y + 1) \pmod{3^{q+1}}. \quad (8)$$

Multiplication by three makes this independent of the representative of $Y \pmod{3^q}$; 2^{-V} denotes a modular inverse. Set

$$\rho_q(y) = \frac{2}{3} 3^q \mu_q(y), \quad \langle g \rangle_q = \frac{1}{3^q} \sum_{y \pmod{3^q}} g(y).$$

The definition gives

$$0 \leq \rho_q(y) \leq \frac{2}{3} 3^q, \quad \langle \rho_q \rangle_q = \frac{2}{3}. \quad (9)$$

The averaging measure is uniform on *all* residues, not only the units. For $k \geq m$ let $\pi_{k,m}$ be reduction modulo 3^m . The fine-scale mixing estimate used here is

$$\exists C \geq 0 \ \forall k \geq m \geq 1 : \quad \langle |\rho_k - \rho_m \circ \pi_{k,m}| \rangle_k \leq \frac{C}{m^2}. \quad (10)$$

This is a cited analytic estimate, not a consequence of (9). The supporting development proves an arbitrary fixed polynomial rate; only exponent two is needed here.

4.2 Inverse-path data and the estimates used

An inverse path ending at an odd seed M is a finite list of odd positive integers $x_0, \dots, x_d = M$ with $S(x_i) = x_{i+1}$. Its valuation word is $(v_0, \dots, v_{d-1})$, where

$$3x_i + 1 = 2^{v_i} x_{i+1}, \quad v_i \geq 1.$$

Once x_0 and d are given, this word is uniquely determined. Conversely, a word and its endpoint determine the path by solving $x_i = (2^{v_i} x_{i+1} - 1)/3$ backwards whenever these are odd integers.

The cited construction organizes such paths into finite weighted families. We state its required properties in reduced mathematical notation for the singleton-seed case. This is a combined statement of several estimates in the supporting development, not a claim that one source theorem has exactly this presentation. The correspondence to source declarations is recorded in the reproducibility appendix.

Theorem 7 (Analytic inverse-path inputs, cited). *Fix the initial scale $b = 32^5$ and the cap $K_n = \lfloor n/100 \rfloor$. The construction of [1] has the following properties.*

1. *For every odd $M \geq 16^b$ it gives a nonnegative sequence $A_n(M)$. There are a number $p > 0$ and nonnegative numbers d_n , all independent of M , such that $\sum_{n \geq 0} d_n < \infty$ and*

$$|A_{n+1}(M) - A_n(M)| \leq d_n. \quad (11)$$

For each n there are a modulus 3^{q_n} and a residue r_n such that every sufficiently large odd $M \equiv r_n \pmod{3^{q_n}}$ satisfies

$$A_n(M) \geq \frac{2}{3}p. \quad (12)$$

The sequence d_n is chosen before the physical seed M .

2. A generation n has a finite label set I_n , roots $r_{n,i} > 0$, and a common scale $b_n \geq b + 2n$ independent of M . For suitable nonnegative integer shift radii s_n , put

$$L_{n,i} = \frac{4^{b_n} r_{n,i}}{4^{2s_n} 3^{b_n}}, \quad U_{n,i} = 2^{2s_n} L_{n,i}.$$

Call $x > 0$ admissible at n if $L_{n,i} < x \leq U_{n,i}$ for every i . For $N \geq 4 \cdot 10^9$, if $x > U_{N,i}$ for every i , then x is admissible at some $n \geq N$.

3. For admissible (n, x) there is a nonnegative terminal mark $B_{n,x}$. If $\ell(M) = \lim_n A_n(M) > 0$, then for some N_0 ,

$$B_{n,x} \geq \ell(M)/2 \quad (n \geq N_0, x \text{ admissible at } n). \quad (13)$$

Write $k_n = \lfloor b_n/4 \rfloor$. Positivity of this mark implies $3^{k_n} \leq x$.

4. For each admissible (n, x) there is a finite family $\mathcal{Z}_{n,x}$ of terminal paths to M , with source $s(z)$, weight $w(z) \geq 0$, and

$$x \leq s(z) < 32x, \quad s(z)w(z) \leq M. \quad (14)$$

Two members with the same valuation word are identical. With $F_j(t) = 4^j t + (4^j - 1)/3$ and $J_n = \lfloor K_n/2 \rfloor + 1$, one has

$$B_{n,x} \leq \sum_{z \in \mathcal{Z}_{n,x}} w(z) \sum_{j=0}^{J_n-1} 4^{-j} \rho_{k_n}(F_j(s(z)) \bmod 3^{k_n}). \quad (15)$$

4.3 An explicit summable majorant

The summability assertion in the first input can be made explicit. For some $C_v \geq 0$ furnished by the cited increment estimate, set

$$g = \frac{2013}{2000}, \quad r_1 = g \left(\frac{200}{201} \right)^6, \quad r_2 = g 2^{-1/100}, \quad H = 8 \cdot 35! \cdot 576^{35}.$$

For the fixed cap $K_n = \lfloor n/100 \rfloor$, one may take

$$d_n = 4(4b^{3/5} + 1)(2^{b+1} + 16^b)(n+1)^3 \times \left[\frac{2C_v 8^6 + H}{b^6} r_1^n + r_2^n \right]. \quad (16)$$

All factors preceding the powers are independent of the root M . To check summability, first note that $(201/200)^6 \geq 1 + 6/200 > 2013/2000$, so $0 < r_1 < 1$. For the other ratio write $g = 1 + 13/2000$ and $t = 13/20$. The binomial theorem and $\binom{100}{j} \leq 100^j/j!$ give

$$g^{100} \leq \sum_{j=0}^{100} \frac{t^j}{j!} \leq 1 + t + \frac{t^2}{2} \sum_{j=0}^{\infty} (t/3)^j = 1 + t + \frac{t^2}{2(1-t/3)} < 2.$$

Here $(j+2)! \geq 2 \cdot 3^j$ bounds the tail, and the last inequality is a rational calculation at $t = 13/20$. Therefore $0 < r_2 < 1$. For either $r \in (0, 1)$ the ratio of consecutive terms of $(n+1)^3 r^n$ tends to $r < 1$, so its series converges. Equation (16) is a nonnegative linear combination of two such summable sequences. This proves the claimed summability; the bound of the actual increments by this majorant remains the cited analytic estimate.

The weights describe labeled paths; they are not assumed to be counts or probabilities on distinct sources. In particular, injectivity of the source map is deliberately absent from the analytic inputs. We prove it below using the nonperiodicity of M .

The principal analytic ingredients behind this statement are finite-residue averaging, a positive lower bound on the retained probability product, a summable variation bound uniform in the initial root, compression of the first valuation, and synchronization of the physical intervals. Their proofs and the proof of (10) are the cited portion of the argument. The following sections give all deductions from those inputs.

5 Choosing one seed with a positive limiting mark

Lemma 8. *If a is odd and $3 \nmid a$, there is a nonperiodic odd M with $S(M) = a$ and $\ell(M) > 0$.*

Proof. Let $D_N = \sum_{n=N}^{\infty} d_n$. Summability gives $D_N \rightarrow 0$. For any fixed allowed seed M and $t > N$, telescoping and (11) give

$$|A_t(M) - A_N(M)| \leq \sum_{n=N}^{t-1} |A_{n+1}(M) - A_n(M)| \leq \sum_{n=N}^{t-1} d_n \leq D_N.$$

Thus $A_n(M)$ is Cauchy, has a real limit $\ell(M)$, and

$$|\ell(M) - A_N(M)| \leq D_N. \quad (17)$$

Choose N such that $D_N < p/3$. Now apply Theorem 6 to the residue r_N modulo 3^{q_N} , using a size bound large enough for $M \geq 16^b$ and (12). This chooses one nonperiodic seed M with $S(M) = a$ and $A_N(M) \geq 2p/3$. Consequently

$$\ell(M) \geq A_N(M) - D_N > \frac{2p}{3} - \frac{p}{3} = \frac{p}{3} > 0.$$

All later generations use this same M . Uniformity of D_N in M is what permits the order of choices p, d_n , then N , then M . $\square$

6 From path weights to distinct sources

Fix the seed M from Lemma 8, and an admissible pair (n, x) . Abbreviate $\mathcal{Z} = \mathcal{Z}_{n,x}$ and put

$$E = \{s(z) : z \in \mathcal{Z}\}, \quad W = \sum_{z \in \mathcal{Z}} w(z).$$

Lemma 9 (Multiplicity and weighted counting). *The map $z \mapsto s(z)$ is injective. For every nonnegative function ψ on the positive integers,*

$$\sum_{z \in \mathcal{Z}} w(z) \psi(s(z)) \leq \frac{M}{x} \sum_{s \in E} \psi(s). \quad (18)$$

In particular $xW \leq M\#E$.

Proof. Suppose two terminal paths have the same source s and lengths d, e . Both hit M . Non-periodicity and Lemma 5 imply $d = e$. The deterministic Syracuse orbit fixes their valuation words, so the terminal-path uniqueness in Theorem 7 makes the members equal. By (14), $xw(z) \leq s(z)w(z) \leq M$. Multiply by $\psi(s(z)) \geq 0$, sum, and use injectivity:

$$x \sum_z w(z) \psi(s(z)) \leq M \sum_z \psi(s(z)) = M \sum_{s \in E} \psi(s).$$

Divide by $x > 0$. Taking $\psi = 1$ proves the final assertion. $\square$

Lemma 10 (Residue census). *If $Q = 3^k \leq x$ and $\psi : \mathbb{Z}/Q\mathbb{Z} \rightarrow \mathbb{R}_{\geq 0}$, then*

$$\sum_{z \in \mathbb{Z}} w(z) \psi(s(z) \bmod Q) \leq 33M \langle \psi \rangle_k. \quad (19)$$

Proof. For any residue, the number of nonnegative integers $t < 32x$ in that residue is at most $32x/Q + 1 \leq 33x/Q$. Since $E \subset [x, 32x)$,

$$\sum_{s \in E} \psi(s \bmod Q) \leq \frac{33x}{Q} \sum_{y \bmod Q} \psi(y) = 33x \langle \psi \rangle_k.$$

Apply (18) and cancel x . □

7 Removing the reference-density mark

The transformation $t \mapsto 4t + 1$ has iterates $F_j(t) = 4^j t + (4^j - 1)/3$. Indeed this holds at $j = 0$, and $4F_j(t) + 1 = F_{j+1}(t)$. Since 4^j is invertible modulo 3^k , each F_j permutes those residues and preserves their uniform mean.

Fix integers $1 \leq m \leq k = k_n$ and suppose $B_{n,x} > 0$. Then $3^k \leq x$ by Theorem 7. Write

$$\delta(y) = |\rho_k(y) - \rho_m(\pi_{k,m}y)|, \quad \varepsilon = \frac{C}{m^2}, \quad H_m = \frac{2}{3} 3^m.$$

Equations (9) and (10) imply

$$\rho_k(y) \leq H_m + \delta(y), \quad \langle \delta \rangle_k \leq \varepsilon.$$

Using the residue census with $\psi = \delta \circ F_j$ gives

$$\begin{aligned} \sum_z w(z) \rho_k(F_j(s(z))) &\leq H_m W + \sum_z w(z) \delta(F_j(s(z))) \\ &\leq H_m W + 33M \langle \delta \circ F_j \rangle_k \\ &= H_m W + 33M \langle \delta \rangle_k \leq H_m W + 33M \varepsilon. \end{aligned}$$

Residues modulo 3^k are implicit in these evaluations. Multiply by 4^{-j} and sum over $0 \leq j < J_n$. Since every summand is nonnegative and $\sum_{j=0}^{J_n-1} 4^{-j} \leq \sum_{j=0}^{\infty} 4^{-j} = 4/3$, (15) yields the complete estimate

$$B_{n,x} \leq \frac{4}{3} (H_m W + 33M \varepsilon) = \frac{8}{9} 3^m W + \frac{44MC}{m^2}. \quad (20)$$

Thus the constants $8/9$ and 44 arise respectively from $(4/3)(2/3)$ and $(4/3)33$.

Let $\alpha = \ell(M)/2 > 0$. Choose an integer

$$m > \max\{1, 88MC/\alpha\}. \quad (21)$$

As $m^2 \geq m$, this implies

$$\frac{44MC}{m^2} \leq \frac{44MC}{m} < \frac{\alpha}{2}$$

when $MC > 0$; if $MC = 0$ the same desired upper bound is immediate. For $n \geq 2m$, the inequality $b_n \geq b + 2n \geq 4m$ gives $k_n \geq m$. For all sufficiently large n the bound (13) also holds. Combining it with (20), for every admissible x we get

$$\alpha \leq B_{n,x} \leq \frac{8}{9} 3^m W + \frac{\alpha}{2}, \quad W \geq \eta := \frac{\alpha}{2(8/9)3^m} > 0.$$

The integer m and the number η have now been fixed independently of both n and x .

8 A lower bound at every sufficiently large cutoff

Choose N large enough for the preceding mass bound, with $N \geq 4 \cdot 10^9$. Since I_N is finite, define

$$x_* = \sum_{i \in I_N} \max\{0, U_{N,i}\}, \quad Y_* = \lceil 32(\max\{x_*, 0\} + 1) \rceil.$$

Take any integer $Y \geq Y_*$ and put $x = Y/32$. Then $x > x_*$ and $x > U_{N,i}$ for every i . Interval synchronization gives some $n \geq N$ at which x is admissible. At this generation $W \geq \eta$. Every source is odd and satisfies $s(z) < 32x = Y$. Its terminal path reaches M , and $S(M) = a$, so $E \subseteq \{s < Y : S^k(s) = a \text{ for some } k\}$. Lemma 9 therefore gives

$$\frac{Y}{32}\eta = x\eta \leq xW \leq M\#E \leq MQ_a(Y).$$

Dividing by $M > 0$ proves

$$Q_a(Y) \geq \frac{\eta}{32M}Y \quad (Y \geq Y_*). \quad (22)$$

The synchronization step is why this is a bound for every sufficiently large Y , rather than for a selected sequence of cutoffs.

9 All positive targets and the divisible-by-three exception

Proof of Theorem 1. First let $a > 0$ and $3 \nmid a$, allowing a to be even. The formulas in Lemma 2 still give an odd positive c and $v \in \{1, 2\}$ with $3c + 1 = 2^v a$. If $3 \nmid c$, set $d = c$ and $w = v$. If $3 \mid c$, set $d = 4c + 1$ and $w = v + 2$. In the latter case d is odd, $d \equiv 1 \pmod{3}$, and

$$3d + 1 = 12c + 4 = 4(3c + 1) = 2^{v+2}a.$$

In both cases d is an odd target coprime to three and $T^{w+1}(d) = a$: the odd step gives $2^w a$ and w halvings give a . Every odd Syracuse predecessor of d is consequently an ordinary predecessor of a . Equation (22), applied to d , gives $P_a(Y) \geq Q_d(Y) \geq c_d Y$ for all sufficiently large Y .

Now suppose $3 \mid a$. We prove by induction on k that $T^k(n) = a$ implies $n = 2^j a$ for some $j \geq 0$. For $k = 0$ take $j = 0$. For $k + 1$, the induction hypothesis applied to $T(n)$ gives $T(n) = 2^j a$, which is divisible by three. An odd n would give $T(n) = 3n + 1 \equiv 1 \pmod{3}$, a contradiction. Hence n is even and $n = 2T(n) = 2^{j+1}a$. Conversely $T^j(2^j a) = a$ by j halvings. Thus

$$R_a = \{2^j a : j \geq 0\}, \quad P_a(2^k a) = k. \quad (23)$$

For completeness, if $2^k a \leq X < 2^{k+1}a$, then $P_a(X) \leq k + 1$ and $P_a(X)/X \leq (k + 1)/(2^k a) \rightarrow 0$. Thus R_a has natural density zero. In particular it cannot satisfy an eventual positive linear lower bound. This proves both directions. $\square$

10 The exact consequence for possible counterexamples

Let $\mathcal{B} = \{n > 0 : n \not\rightarrow 1\}$ and $B(X) = \#(\mathcal{B} \cap [0, X))$.

Lemma 11. *If $\mathcal{B} \neq \emptyset$, there is $a \in \mathcal{B}$ with $3 \nmid a$, and $R_a \subseteq \mathcal{B}$.*

Proof. Starting from $n \in \mathcal{B}$, divide out its factors of two and reach an odd u . If $3 \nmid u$, take $a = u$; otherwise take $a = 3u + 1$. Then $3 \nmid a$ and $n \rightarrow a$. The point a cannot reach one, as concatenating paths would make n reach one.

Suppose $s \rightarrow a$ and $s \rightarrow 1$, with hitting times d, t respectively. If $d \leq t$, then $T^{t-d}(a) = 1$. If $t < d$, then $a = T^{d-t}(1)$ lies on the cycle $1, 4, 2$, each point of which reaches one. Both possibilities contradict $a \in \mathcal{B}$. Thus every $s \in R_a$ lies in $\mathcal{B}$. $\square$

Theorem 12 (Counterexample density and a conditional criterion). *If $\mathcal{B}$ is nonempty, then $B(X) \geq cX$ eventually for some $c > 0$. Moreover universal Collatz convergence is equivalent to*

$$\forall \varepsilon > 0 \ \forall X_0 \in \mathbb{N} \ \exists X \in \mathbb{N} : \quad X \geq X_0, \quad X > 0, \quad B(X) < \varepsilon X. \quad (24)$$

Proof. Choose a by Lemma 11. Theorem 1 and inclusion give $B(X) \geq P_a(X) \geq c_a X$ for all sufficiently large X . If every positive integer converges, $\mathcal{B} = \emptyset$ and (24) holds with $X = \max\{X_0, 1\}$. Conversely assume (24). If a counterexample existed, the first part would supply $c > 0$ and a cutoff X_0 with $B(X) \geq cX$ for all $X \geq X_0$. Taking $\varepsilon = c$ in (24) would give $B(X) < cX$ at such a cutoff, a contradiction. $\square$

No proof of the hypothesis (24) is supplied. The theorem therefore gives a conditional criterion, not a proof of universal convergence.

References

- [1] L. Mazur, *Explicit Positive-Density Collatz Convergence in Logarithmic Time*, manuscript v2.1, 6 September 2026, with accompanying formal source development. ProofAtlas. <https://www.proofatlas.ai/formalizations/positive-density-log-time-collatz/>.
- [2] L. Mazur, *Positive Lower Density of Collatz Predecessors*, manuscript v1.0, 6 September 2026. ProofAtlas. <https://www.proofatlas.ai/formalizations/positive-lower-density-collatz-predecessors/>.

A Verification and reproducibility

The verification run completed successfully on 12 September 2026. The toolchain was Lean 4.30.0-rc2. Run from the project directory:

```
LEAN_NUM_THREADS=4 python3 scripts/verify.py
```

The script reported **verified**. It checked 1,485 upstream source entries against their manifest hashes, checked nine dependency revisions against the upstream lock file, recorded local source SHA-256 hashes, and completed the build of both libraries (5,105 build jobs, including replayed jobs). It then audited 255 listed theorem endpoints. The public target-density theorem and counterexample dichotomy use only **propect**, **Classical.choice**, and **Quot.sound**. No audited endpoint used an axiom outside this standard set. The source escape scan also passed; in particular the scanned proof sources contain no executable **sorry** or additional axiom declaration.

The machine-readable record is **verification/local-report.json**; build and axiom logs are in the same directory. Every one of the 13 local modules reproduced below was part of this build. The appendix contains 1,283 source lines and was compared byte-for-byte with those modules. It relies on the project’s pinned imported libraries, so compilation of isolated snippets without those imports is not the verification procedure.

To regenerate this PDF from its output directory, run XeLaTeX twice:

```
xelatex -interaction=nonstopmode -halt-on-error
↪ collatz_target_density_report.tex
xelatex -interaction=nonstopmode -halt-on-error
↪ collatz_target_density_report.tex
```

The adjacent **fonts/JuliaMono-Regular.ttf** supplies the Unicode characters in the Lean listings.

The check validates the formal statements under Lean’s foundations and the installed compiler and dependencies. It does not by itself establish historical novelty or replace expert review of the translation from an informal claim to its definitions. The public map and count definitions are printed verbatim below so that this translation can be inspected directly.

The source revision used by this local development is

```
0aef8b0cfaaf6959500063472f2e328c60df8d54
```

This identifier records the local dependency version. It is not a mathematical hypothesis or a replacement for the references above. The current public source release may use a different packaging revision.

A.1 Correspondence for the cited analytic inputs

The combined input theorem in the main text specializes the construction to one initial root of weight one. Its parent source potential is therefore exactly M and its denominator is one. The following declarations identify the precise imported ingredients. All lie in the accompanying supporting source tree. The displayed theorem is an expository combination of these results; it is not a verbatim transcription of a single Lean declaration.

The probability law and reference normalization are defined by:

```
Erdos1135.Tao.syracPMF
Erdos1135.ND.PositiveDensity.ndSyracuseUnitReferenceDensity
```

The fine-scale estimate is:

```
unitReferenceDensity_fulll1_polynomial_rate
```

The finite residue average, its realization by a physical root, and the positive probability product use:

```
exists_unit_rootCoreBackwardMark_ge_product
rootCoreSingletonState_markedMass_eq_backwardMark
exists_pos_le_coreProbabilityProduct
```

The summable root-uniform increment bound and its convergence consequences use:

```
forward_core_increment_le_majorant
summable_rootCoreVariationMajorant
exists_coreMarkedLimit_with_root_uniform_tail
```

The terminal lower bound and the room for the modulus use:

```
eventually_terminalUnitMass_ge_of_pos_core_limit
terminal_conductor_room_of_pos_mark
```

The interval, word uniqueness, source-weight budget, and compression inputs use:

```
exists_unstopped_generation_all_intervals_contain
fullTerminal_eq_of_ancestor_word_eq
fullTerminal_source_mul_weight_le_frozenOwner
forwardCoreTerminalUnitMass_le_full_cap_one_fan
ndGeom2RootSideCommonFloor_capOne_source_bounds
```

The local code below proves the adaptations using nonperiodicity, including the weighted count, residue census, two-scale estimate, and final assembly. The abstract data in the main text denote these concrete path families and marks; no replacement probability model or unproved Collatz assumption is introduced.

B Lean source: complete local dependency chain

The following files are reproduced verbatim from the verified project, in dependency order, starting from the public theorem and the counterexample dichotomy. They compile as modules within that project, with its pinned imports; the listings are not a single standalone Lean file. This appendix includes every transitive local module needed by those two endpoints. The much larger imported upstream and Mathlib libraries remain external.

B.1 CollatzTargetDensity/Dynamics.lean

```
1 import Mathlib.Logic.Function.Iterate
2 import Lean.Elab.Tactic.Omega
3
4 /-!+# Nonperiodic seed selection
5
6 Elementary deterministic dynamics used by the proposed all-target extension.
7 No Collatz convergence hypothesis occurs in these statements.
8 -/
9
10 namespace CollatzTargetDensity
11
12 def Nonperiodic {α : Type*} (f : α → α) (x : α) : Prop :=
13   ∀ k : ℕ, 0 < k → (f^[k]) x ≠ x
14
15 theorem periodic_multiples {α : Type*} {f : α → α} {x : α} {p : ℕ}
16   (hp : (f^[p]) x = x) (q : ℕ) : (f^[q * p]) x = x := by
17   induction q with
18   | zero => simp
19   | succ q ih => rw [Nat.succ_mul, Function.iterate_add_apply, hp, ih]
20
21 /-- The restriction of any function to its periodic points is injective. -/
22 theorem periodic_predecessors_eq {α : Type*} {f : α → α} {x y : α}
23   {p q : ℕ} (hp : 0 < p) (hq : 0 < q)
24   (hpx : (f^[p]) x = x) (hpy : (f^[q]) y = y)
```

```

25   (hxy : f x = f y) : x = y := by
26   have hprod : 0 < q * p := Nat.mul_pos hq hp
27   have hx := periodic_multiples hpx q
28   have hy : (f^[q * p]) y = y := by
29     simpa only [Nat.mul_comm] using periodic_multiples hqy p
30   have he : q * p = (q * p - 1) + 1 := by omega
31   have hiter : (f^[q * p]) x = (f^[q * p]) y := by
32     rw [he, Function.iterate_add_apply, Function.iterate_add_apply]
33     simp only [Function.iterate_one, hxy]
34   exact hx.symm.trans (hiter.trans hy)
35
36 /-- Two distinct immediate predecessors provide a nonperiodic choice. -/
37 theorem nonperiodic_choice {α : Type*} {f : α → α} {x y : α}
38   (hne : x ≠ y) (hxy : f x = f y) : Nonperiodic f x ∨ Nonperiodic f y := by
39   classical
40   by_cases hx : Nonperiodic f x
41   · exact Or.inl hx
42   · right
43     obtain ⟨p, hp, hpx⟩ : ∃ p, ∃ _ : 0 < p, (f^[p]) x = x := by
44       simpa only [Nonperiodic, not_forall, not_not] using hx
45     intro q hq hqy
46     exact hne (periodic_predecessors_eq hp hq hpx hqy hxy)
47
48 /-- Repeated hits would make the target itself periodic. -/
49 theorem nonperiodic_hit_time_unique {α : Type*} {f : α → α} {source target : α}
50   (ht : Nonperiodic f target) {d e : ℕ}
51   (hd : (f^[d]) source = target) (he : (f^[e]) source = target) : d = e := by
52   have hnot (a b : ℕ) (ha : (f^[a]) source = target)
53     (hb : (f^[b]) source = target) (hab : a < b) : False := by
54     have hsplit : b = (b - a) + a := by omega
55     rw [hsplit, Function.iterate_add_apply, ha] at hb
56     exact ht (b - a) (by omega) hb
57   by_cases hde : d < e
58   · exact False.elim (hnot d e hd he hde)
59   · by_cases hed : e < d
60     · exact False.elim (hnot e d he hd hed)
61     · omega
62
63 /-- A residue-wise supply of two distinct children is enough; neither child's
64   eventual convergence is required. The predicate records residue and size. -/
65 theorem select_nonperiodic_child {α : Type*} {f : α → α} (target : α)
66   (P : α → Prop)
67   (hchildren : ∃ x y, P x ∧ P y ∧ f x = target ∧ f y = target ∧ x ≠ y) :
68   ∃ x, P x ∧ f x = target ∧ Nonperiodic f x := by
69   obtain ⟨x, y, hx, hy, hfx, hfy, hne⟩ := hchildren
70   rcases nonperiodic_choice hne (hfx.trans hfy.symm) with h | h
71   · exact ⟨x, hx, hfx, h⟩
72   · exact ⟨y, hy, hfy, h⟩
73
74 end CollatzTargetDensity

```

B.2 CollatzTargetDensity/SeedSupply.lean

```

1 import CollatzTargetDensity.Dynamics
2 import Erdos1135.Tao.Syracuse.Basic
3
4 namespace CollatzTargetDensity
5 open Erdos1135
6
7 /-- The arithmetic input to the analytic argument: every finite ternary class
8   contains arbitrarily large nonperiodic immediate predecessors. -/
9 def SeedSupply (target : ℕ) : Prop :=
10   ∀ q X : ℕ, ∀ r : Fin (3 ^ q), ∃ M : ℕ,
11     Odd M ∧ X < M ∧ Tao.syracuse M = target ∧ M % 3 ^ q = r ∧
12     Nonperiodic Tao.syracuse M

```

```

13
14 end CollatzTargetDensity

```

B.3 CollatzTargetDensity/ResidueSeedCore.lean

```

1 import Erdos1135.ND.PositiveDensity.A5HistoricalAdjacentCylinderPreviousPhysicalLowerShellNoGo
2 import Mathlib.NumberTheory.Multiplicity
3
4 /-!
5 # Arithmetic residue coverage, isolated from density imports
6
7 The following proofs are copied without mathematical change from Lech Mazur's
8 Apache-2.0 source package, corrected commit
9 0aef8b0cfaaf6959500063472f2e328c60df8d54,
10 'SuccessfulRootTernaryResidueCoverage.lean'. Modification: namespace changed and
11 unneeded target-counting imports and later analytic declarations omitted.
12 This is known arithmetic, not a claimed new result.
13 -/
14 namespace CollatzTargetDensity.ResidueSeedCore
15 open Erdos1135 Erdos1135.ND.PositiveDensity
16
17 theorem ndCollapseSource_add (m n : ℕ) :
18   ndA5PreviousPhysicalCollapseSource (m + n) =
19     ndA5PreviousPhysicalCollapseSource m + 4 ^ m * ndA5PreviousPhysicalCollapseSource n :=
20   ↪ by
21   have ha := three_mul_ndA5PreviousPhysicalCollapseSource_add_one (m + n)
22   have hm := three_mul_ndA5PreviousPhysicalCollapseSource_add_one m
23   have hn := three_mul_ndA5PreviousPhysicalCollapseSource_add_one n
24   rw [pow_add, ← hm, ← hn] at ha
25   nlinarith
26
27 theorem threePow_succ_dvd_fourPow_sub_one_iff (q n : ℕ) :
28   3 ^ (q + 1) | 4 ^ n - 1 ↔ 3 ^ q | n := by
29   rw [pow_dvd_iff_le_emultiplicity, pow_dvd_iff_le_emultiplicity]
30   have h := Nat.emultiplicity_pow_sub_pow
31   (p := 3) (x := 4) (y := 1)
32   (by norm_num : Nat.Prime 3) (by norm_num : Odd 3)
33   (by norm_num : 3 | 4 - 1) (by norm_num : ¬ 3 | 4) n
34   simp only [one_pow] at h
35   rw [h]
36   norm_num [Nat.Prime.emultiplicity_self (by norm_num : Nat.Prime 3)]
37   rw [add_comm (1 : ENat), ENat.add_le_add_iff_right (by simp)]
38
39 theorem threePow_dvd_collapseSource_iff (q n : ℕ) :
40   3 ^ q | ndA5PreviousPhysicalCollapseSource n ↔ 3 ^ q | n := by
41   have h := three_mul_ndA5PreviousPhysicalCollapseSource_add_one n
42   have he : 4 ^ n - 1 = 3 * ndA5PreviousPhysicalCollapseSource n := by omega
43   have hl := threePow_succ_dvd_fourPow_sub_one_iff q n
44   rw [he, pow_succ, Nat.mul_comm (3 ^ q) 3,
45     Nat.mul_dvd_mul_iff_left (by norm_num : 0 < 3)] at hl
46   exact hl
47
48 theorem ndCollapseSource_modEq_of_add_iff (q m n : ℕ) :
49   Nat.ModEq (3 ^ q) (ndA5PreviousPhysicalCollapseSource m)
50     (ndA5PreviousPhysicalCollapseSource (m + n)) ↔ 3 ^ q | n := by
51   rw [ndCollapseSource_add, Nat.left_modEq_add_iff]
52   have hc : Nat.Coprime (3 ^ q) (4 ^ m) := by
53     exact (show Nat.Coprime 3 4 by norm_num).pow q m
54   rw [hc.dvd_mul_left, threePow_dvd_collapseSource_iff]
55
56 theorem ndCollapseSource_residue_injective (q : ℕ) :
57   Function.Injective (fun m : Fin (3 ^ q) =>
58     ((ndA5PreviousPhysicalCollapseSource m % 3 ^ q,
59       Nat.mod_lt _ (by positivity)) : Fin (3 ^ q))) := by
60   intro m n he

```

```

60 apply Fin.ext
61 have heq : Nat.ModEq (3 ^ q) (ndA5PreviousPhysicalCollapseSource m)
62   (ndA5PreviousPhysicalCollapseSource n) := congrArg Fin.val he
63 suffices h : ∀ (a b : Fin (3 ^ q)), a ≤ b →
64   Nat.ModEq (3 ^ q) (ndA5PreviousPhysicalCollapseSource a)
65   (ndA5PreviousPhysicalCollapseSource b) → (a : ℕ) = b by
66   rcases le_total m n with hmn | hnm
67   · exact h m n hmn heq
68   · exact (h n m hnm heq.symm).symm
69 intro a b hab hmod
70 obtain ⟨d, hd⟩ := Nat.exists_eq_add_of_le (show (a : ℕ) ≤ b from hab)
71 rw [hd] at hmod
72 have hdiv := (ndCollapseSource_modEq_of_add_iff q a d).mp hmod
73 have hdl : d < 3 ^ q := by have h := b.isLt; omega
74 have hz : d = 0 := Nat.eq_zero_of_dvd_of_lt hdiv hdl
75 omega
76
77 /-- The existing successful family is a permutation of every finite
78 ternary residue space. No finite modulus range is enumerated. -/
79 theorem ndCollapseSource_residue_surjective (q : ℕ) :
80   Function.Surjective (fun m : Fin (3 ^ q) =>
81     (⟨ndA5PreviousPhysicalCollapseSource m % 3 ^ q,
82       Nat.mod_lt _ (by positivity)⟩ : Fin (3 ^ q))) :=
83   Finite.surjective_of_injective (ndCollapseSource_residue_injective q)
84
85 /-- **ARBITRARILY HIGH SUCCESSFUL RESIDUE COVERAGE.** Every ternary class
86 contains unbounded odd integers that reach one in one Syracuse step. -/
87 theorem exists_large_oneStepSuccessful_root_in_residue
88   (q X : ℕ) (r : Fin (3 ^ q)) :
89   ∃ M : ℕ, X < M ∧ Odd M ∧ Tao.syracuse M = 1 ∧ M % 3 ^ q = r := by
90   obtain ⟨m, hm⟩ := ndCollapseSource_residue_surjective q r
91   let k := (m : ℕ) + (X + 1) * 3 ^ q
92   have hQ : 1 ≤ 3 ^ q := by
93     have h : 0 < 3 ^ q := by positivity
94     omega
95   have hXk : X < k := by
96     have h := Nat.mul_le_mul_left (X + 1) hQ
97     dsimp only [k]
98     omega
99   have hmod : Nat.ModEq (3 ^ q) (ndA5PreviousPhysicalCollapseSource m)
100     (ndA5PreviousPhysicalCollapseSource k) :=
101     (ndCollapseSource_modEq_of_add_iff q m ((X + 1) * 3 ^ q)).2
102     (Nat.dvd_mul_left _ _)
103   have hr : ndA5PreviousPhysicalCollapseSource k % 3 ^ q = r := by
104     change ndA5PreviousPhysicalCollapseSource m % 3 ^ q =
105       ndA5PreviousPhysicalCollapseSource k % 3 ^ q at hmod
106     exact hmod.symm.trans (congrArg Fin.val hm)
107   obtain ⟨j, hj⟩ := Nat.exists_eq_succ_of_ne_zero (by omega : k ≠ 0)
108   refine ⟨ndA5PreviousPhysicalCollapseSource k,
109     hXk.trans_le (self_le_ndA5PreviousPhysicalCollapseSource k), ?_, ?_, hr⟩
110   · rw [hj]
111     exact ndA5PreviousPhysicalCollapseSource_odd j
112   · rw [hj]
113     exact syracuse_ndA5PreviousPhysicalCollapseSource_succ j
114
115
116 end CollatzTargetDensity.ResidueSeedCore

```

B.4 CollatzTargetDensity/ArithmeticSeeds.lean

```

1 import CollatzTargetDensity.SeedSupply
2 import CollatzTargetDensity.ResidueSeedCore
3
4 /-!+## Arbitrary-target nonperiodic seed supply
5

```

```

6 The residue coverage is the classical backward-mixing phenomenon. The proof
7 below uses the imported successful-root coverage as an arithmetic parameter
8 space, then transports it to another target by an affine map. It does not
9 assert that the new target converges to 1.
10 -/
11
12 namespace CollatzTargetDensity
13 open Erdos1135 Erdos1135.ND.PositiveDensity
14 noncomputable section
15
16 theorem exists_odd_child {target : ℕ} (hodd : Odd target) (hthree : target % 3 ≠ 0) :
17   ∃ c : ℕ, Odd c ∧ Tao.syracuse c = target := by
18   have hnotdvd : ¬2 ∣ target := by
19     intro hdiv
20     exact (Nat.not_even_iff_odd.mpr hodd) (even_iff_two_dvd.mpr hdiv)
21   by_cases h : target % 3 = 1
22   · let c := (4 * target - 1) / 3
23     have he : 3 * c + 1 = 4 * target := by dsimp [c]; omega
24     refine ⟨c, ⟨c / 2, by omega⟩, ?_⟩
25     rw [Tao.syracuse_eq_ordCompl_two, he]
26     exact Nat.ordCompl_pow_mul_of_not_dvd 2 Nat.prime_two hnotdvd
27   · let c := (2 * target - 1) / 3
28     have he : 3 * c + 1 = 2 * target := by dsimp [c]; omega
29     refine ⟨c, ⟨c / 2, by omega⟩, ?_⟩
30     rw [Tao.syracuse_eq_ordCompl_two, he]
31     exact Nat.ordCompl_pow_mul_of_not_dvd 1 Nat.prime_two hnotdvd
32
33 /-- The affine transform  $x \mapsto (3c+1)x+c$  takes one-step predecessors of 1
34 to one-step predecessors of  $S(c)$ , and permutes every ternary residue space. -/
35 theorem large_child_in_residue {target c : ℕ} (hc : Odd c)
36   (hct : Tao.syracuse c = target) (q X : ℕ) (r : Fin (3 ^ q)) :
37   ∃ M : ℕ, Odd M ∧ X < M ∧ Tao.syracuse M = target ∧ M % 3 ^ q = r := by
38   have hcop : Nat.Coprime (3 * c + 1) 3 := by
39     rw [Nat.add_comm, Nat.coprime_add_mul_left_left]
40     norm_num
41   have huni : IsUnit ((3 * c + 1 : ℕ) : ZMod (3 ^ q)) := by
42     apply (ZMod.isUnit_iff_coprime _ _).2
43     simp using hcop.pow 1 q
44   let f : ZMod (3 ^ q) → ZMod (3 ^ q) :=
45     fun x => ((3 * c + 1 : ℕ) : ZMod (3 ^ q)) * x + c
46   have hinj : Function.Injective f := by
47     intro x y hxy
48     exact huni.mul_left_cancel (add_right_cancel hxy)
49   obtain ⟨y, hy⟩ := Finite.surjective_of_injective hinj (r.val : ZMod (3 ^ q))
50   obtain ⟨x, hx, _hxodd, hxhit, hxres⟩ :=
51     ↪ ResidueSeedCore.exists_large_oneStepSuccessful_root_in_residue
52     q X ⟨y.val, ZMod.val_lt y⟩
53   let M := (3 * c + 1) * x + c
54   have hxcast : (x : ZMod (3 ^ q)) = y := by
55     calc
56     _ = ((x % 3 ^ q : ℕ) : ZMod (3 ^ q)) := by simp
57     _ = ((y.val : ℕ) : ZMod (3 ^ q)) := congrArg (fun a : ℕ => (a : ZMod (3 ^ q))) hxres
58     _ = y := ZMod.natCast_zmod_val y
59   have hMcast : (M : ZMod (3 ^ q)) = (r.val : ZMod (3 ^ q)) := by
60     simp only [M, f, Nat.cast_add, Nat.cast_mul, Nat.cast_ofNat, hxcast] using hy
61   refine ⟨M, ?_, ?_, ?_, ?_⟩
62   · obtain ⟨k, hk⟩ := hc
63     refine ⟨(3 * k + 2) * x + k, ?_⟩
64     dsimp [M]
65     rw [hk]
66     ring
67   · dsimp [M]
68     nlinarith
69   · have he : 3 * M + 1 = (3 * c + 1) * (3 * x + 1) := by dsimp [M]; ring
70     rw [Tao.syracuse_eq_ordCompl_two, he, Nat.ordCompl_mul,

```

```

70     ← Tao.syracuse_eq_ordCompl_two, ← Tao.syracuse_eq_ordCompl_two,
71     hct, hxhit, mul_one]
72   · have hv := congrArg ZMod.val hMcast
73     simp only [ZMod.val_natCast, Nat.mod_eq_of_lt r.isLt] using hv
74
75 /-- No unproved dynamical assumption: two distinct children in the same
76 residue class allow exclusion of the unique possible periodic child. -/
77 theorem seed_supply {target : ℕ} (hodd : Odd target) (hthree : target % 3 ≠ 0) :
78   SeedSupply target := by
79   obtain ⟨c, hc, hct⟩ := exists_odd_child hodd hthree
80   intro q X r
81   obtain ⟨x, hxodd, hxlarge, hxhit, hxres⟩ := large_child_in_residue hc hct q X r
82   obtain ⟨y, hyodd, hylarge, hyhit, hyres⟩ := large_child_in_residue hc hct q x r
83   rcases nonperiodic_choice (Nat.ne_of_lt hylarge) (hxhit.trans hyhit.symm) with hx | hy
84   · exact ⟨x, hxodd, hxlarge, hxhit, hxres, hx⟩
85   · exact ⟨y, hyodd, hxlarge.trans hylarge, hyhit, hyres, hy⟩
86
87 end
88 end CollatzTargetDensity

```

B.5 CollatzTargetDensity/SourceCharge.lean

```

1 import CollatzTargetDensity.Dynamics
2 import Erdos1135.ND.PositiveDensity.GeoM2ShiftedWideSymmetricTerminalWeightedCensus
3
4 /-!+## Source charging for nonperiodic seeds
5
6 Adapted from Lech Mazur's Apache-2.0 positive-density source package, corrected
7 commit 0aef8b0cfaaf6959500063472f2e328c60df8d54. The counting inequalities and
8 their proofs are retained; the dynamical hypothesis is changed from convergence
9 to 1 to nonperiodicity. These adaptations are separate from the upstream files.
10 -/
11
12 namespace Erdos1135.ND.PositiveDensity
13 open scoped BigOperators
14 open CollatzTargetDensity
15 noncomputable section
16 namespace NDGeoM2ShiftedWideSymmetricRootSideUniformFloorState
17
18 theorem fullTerminal_eq_of_nonperiodic_ancestor_source_eq
19   {U : NDGeoM2ShiftedWideSymmetricRootSideUniformFloorState}
20   {cap : ℕ → ℕ} {n : ℕ} {shift : (U.forwardIterate cap n).state.Label → ℕ} {K : ℕ}
21   (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))
22   {z w : U.FullTerminalAt cap n shift K}
23   (ha : fullTerminalAncestor z = fullTerminalAncestor w)
24   (hs : ndGeoM2PredictableRootSideBoundedOvershootIncidenceSource z =
25     ndGeoM2PredictableRootSideBoundedOvershootIncidenceSource w) : z = w := by
26   let p := U.fullTerminalPath cap n shift K z
27   let q := U.fullTerminalPath cap n shift K w
28   have hq : (Tao.syracuse^[q.depth])
29     (ndGeoM2PredictableRootSideBoundedOvershootIncidenceSource z) =
30     U.state.root (fullTerminalAncestor z) := by rw [hs, ha]; exact q.terminal_eq hq
31   have hd := nonperiodic_hit_time_unique (hseed _) p.terminal_eq hq
32   have hw := NDGeoM2RootSideSyracusePath.word_eq_of_source_depth_eq p q hs hd
33   have hpw := U.fullTerminalPath_word_eq_reverse cap n shift K z
34   have hqw := U.fullTerminalPath_word_eq_reverse cap n shift K w
35   apply fullTerminal_eq_of_ancestor_word_eq ha
36   apply List.reverse_injective
37   exact hpw.symm.trans (hw.trans hqw)
38
39 theorem fullTerminal_mass_mul_lower_le_frozenPotential_mul_card_of_nonperiodic
40   (U : NDGeoM2ShiftedWideSymmetricRootSideUniformFloorState)
41   (cap : ℕ → ℕ) (n : ℕ)
42   (shift : (U.forwardIterate cap n).state.Label → ℕ) (K : ℕ)
43   (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))

```

```

44 (X : R)
45 (hX : ∀ z : U.FullTerminalAt cap n shift K,
46   X ≤ (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : R)) :
47 X * U.fullTerminalMass cap n shift K ≤
48   U.parentSourcePotential * (U.fullTerminalSources cap n shift K).card := by
49 classical
50 let I := U.state.labelFintype
51 let I := (U.forwardIterate cap n).state.labelFintype
52 let W := fun z : U.FullTerminalAt cap n shift K =>
53   ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight
54   (U.forwardIterate cap n).state.outerWeight z
55 let P := fun i : U.state.Label => (U.state.root i : R) * U.state.outerWeight i
56 let cell := fun z : U.FullTerminalAt cap n shift K =>
57   (fullTerminalAncestor z, ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z)
58 let sources := U.fullTerminalSources cap n shift K
59 have hinj : Function.Injective cell := by
60   intro z w h
61   exact fullTerminal_eq_of_nonperiodic_ancestor_source_eq hseed
62   (congrArg Prod.fst h) (congrArg Prod.snd h)
63 have hsub : Finset.univ.image cell ⊆ Finset.univ.product sources := by
64   intro c hc
65   obtain ⟨z, hz, rfl⟩ := Finset.mem_image.mp hc
66   exact Finset.mem_product.mpr ⟨Finset.mem_univ _, Finset.mem_image.mpr ⟨z, hz, rfl⟩⟩
67 calc
68   X * U.fullTerminalMass cap n shift K = ∑ z, X * W z := by
69     change X * (∑ z, W z) = _
70     rw [Finset.mul_sum]
71   _ ≤ ∑ z : U.FullTerminalAt cap n shift K, P (fullTerminalAncestor z) := by
72     apply Finset.sum_le_sum
73     intro z _
74     have hw := ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight_nonneg
75       (U.forwardIterate cap n).state.outerWeight
76       (U.forwardIterate cap n).state.weight_nonneg z
77     exact (mul_le_mul_of_nonneg_right (hX z) hw).trans
78       (U.fullTerminal_source_mul_weight_le_frozenOwner cap n shift K z)
79   _ = ∑ c ∈ Finset.univ.image cell, P c.1 := by
80     rw [Finset.sum_image]
81     exact fun a _ b _ h => hinj h
82   _ ≤ ∑ c ∈ Finset.univ.product sources, P c.1 := by
83     apply Finset.sum_le_sum_of_subset_of_nonneg hsub
84     intro c _ _
85     exact mul_nonneg (Nat.cast_nonneg _) (U.state.weight_nonneg c.1)
86   _ = U.parentSourcePotential * (U.fullTerminalSources cap n shift K).card := by
87     calc
88       _ = ∑ i : U.state.Label, ∑ _s ∈ sources, P i := Finset.sum_product _ _ _
89       _ = _ := ?_
90   simp only [Finset.sum_const, nsmul_eq_mul, ← Finset.mul_sum]
91   unfold parentSourcePotential ndGeom2PredictableRootSideParentSourcePotential
92   NDGeom2ShiftedWideSymmetricRegenerativeState.rootSideUniformFloorAllLabels
93   dsimp only [P, sources]
94   ring
95
96
97 theorem fullTerminal_weighted_source_charge_of_nonperiodic
98   (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
99   (cap : ℕ → ℕ) (n : ℕ)
100   (shift : (U.forwardIterate cap n).state.Label → ℕ) (K : ℕ)
101   (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))
102   (X : R) (hX : 0 < X)
103   (hsource : ∀ z : U.FullTerminalAt cap n shift K,
104     X ≤ (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : R))
105   (psi : ℕ → R) (hp : ∀ x, 0 ≤ psi x) :
106   let I := (U.forwardIterate cap n).state.labelFintype
107   (∑ z : U.FullTerminalAt cap n shift K,
108     ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight

```

```

109      (U.forwardIterate cap n).state.outerWeight z *
110      psi (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z)) ≤
111      (U.parentSourcePotential / X) *
112       $\sum x \in U.fullTerminalSources \text{ cap } n \text{ shift } K, \text{ psi } x := \text{by}$ 
113      classical
114      letI := U.state.labelFintype
115      letI := (U.forwardIterate cap n).state.labelFintype
116      let W := fun z : U.FullTerminalAt cap n shift K =>
117        ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight
118        (U.forwardIterate cap n).state.outerWeight z
119      let P := fun i : U.state.Label => (U.state.root i : ℝ) * U.state.outerWeight i
120      let cell := fun z : U.FullTerminalAt cap n shift K =>
121        (fullTerminalAncestor z, ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z)
122      let sources := U.fullTerminalSources cap n shift K
123      have hinj : Function.Injective cell := by
124        intro z w h
125        exact fullTerminal_eq_of_nonperiodic_ancestor_source_eq hseed
126        (congrArg Prod.fst h) (congrArg Prod.snd h)
127      have hsub : Finset.univ.image cell ⊆ Finset.univ.product sources := by
128        intro c hc
129        obtain ⟨z, hz, rfl⟩ := Finset.mem_image.mp hc
130        exact Finset.mem_product.mpr ⟨Finset.mem_univ _, Finset.mem_image.mpr ⟨z, hz, rfl⟩⟩
131      have hmul :  $X * (\sum z, W z * \text{psi } (\text{cell } z).2) \leq$ 
132        U.parentSourcePotential *  $\sum x \in \text{sources}, \text{ psi } x := \text{by}$ 
133        calc
134        _ =  $\sum z, (X * W z) * \text{psi } (\text{cell } z).2 := \text{by rw [Finset.mul\_sum]; simp [mul\_assoc]}$ 
135        _ ≤  $\sum z, P (\text{cell } z).1 * \text{psi } (\text{cell } z).2 := \text{by}$ 
136          apply Finset.sum_le_sum
137          intro z _
138          apply mul_le_mul_of_nonneg_right _ (hp _)
139          exact (mul_le_mul_of_nonneg_right (hsource z)
140            (U.forwardIterate cap n).state.weight_nonneg z)).trans
141            (U.fullTerminal_source_mul_weight_le_frozenOwner cap n shift K z)
142        _ =  $\sum c \in \text{Finset.univ.image cell}, P c.1 * \text{psi } c.2 := \text{by}$ 
143          rw [Finset.sum_image]; exact fun a _ b _ h => hinj h
144        _ ≤  $\sum c \in \text{Finset.univ.product sources}, P c.1 * \text{psi } c.2 := \text{by}$ 
145          apply Finset.sum_le_sum_of_subset_of_nonneg hsub
146          intro c _ _
147          exact mul_nonneg (mul_nonneg (Nat.cast_nonneg _) (U.state.weight_nonneg c.1)) (hp _)
148        _ = _ := by
149          calc
150          _ =  $\sum i : U.state.Label, \sum x \in \text{sources}, P i * \text{psi } x := \text{Finset.sum\_product } \_ \_ \_$ 
151          _ = _ := by
152            simp only [← Finset.mul_sum, ← Finset.sum_mul]
153            unfold parentSourcePotential ndGeom2PredictableRootSideParentSourcePotential
154            NDGeom2ShiftedWideSymmetricRegenerativeState.rootSideUniformFloorAllLabels
155            rfl
156      have hdiv := (le_div_iff₀ hX).2 (by simpa [mul_comm] using hmul)
157      simpa [W, cell, sources, div_eq_mul_inv, mul_comm, mul_left_comm, mul_assoc] using hdiv
158
159
160
161 theorem fullTerminal_weighted_residue_census_of_nonperiodic
162   (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
163   (cap : ℕ → ℕ) (n : ℕ)
164   (shift : (U.forwardIterate cap n).state.Label → ℕ) (K : ℕ)
165   (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))
166   (q : ℕ) (X : ℝ) (hX : 0 < X) (hQ : ((3 ^ q : ℕ) : ℝ) ≤ X)
167   (hsource : ∀ z : U.FullTerminalAt cap n shift K,
168     X ≤ (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) ∧
169     (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) < 32 * X)
170   (psi : ZMod (3 ^ q) → ℝ) (hp : ∀ y, 0 ≤ psi y) :
171   letI := (U.forwardIterate cap n).state.labelFintype
172   ( $\sum z : U.FullTerminalAt \text{ cap } n \text{ shift } K,$ 
173     ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight

```

```

174      (U.forwardIterate cap n).state.outerWeight z *
175      psi (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ZMod (3 ^ q))) ≤
176      33 * U.parentSourcePotential * ndTernaryUniformMean q psi := by
177      classical
178      letI := (U.forwardIterate cap n).state.labelFintype
179      have hc := U.fullTerminal_weighted_source_charge_of_nonperiodic cap n shift K hseed
180      X hX (fun z => (hsource z).1) (fun x => psi (x : ZMod (3 ^ q))) (fun x => hp _)
181      have hs := terminal_source_residue_sum_le (U.fullTerminalSources cap n shift K) q X hX hQ
182      (fun x hX => by obtain ⟨z, _, rfl⟩ := Finset.mem_image.mp hx; exact (hsource z).2) psi hp
183      calc
184      _ ≤ _ := hc
185      _ ≤ (U.parentSourcePotential / X) * (33 * X * ndTernaryUniformMean q psi) :=
186      mul_le_mul_of_nonneg_left hs (div_nonneg U.frozenSourcePotential_nonneg hX.le)
187      _ = _ := by field_simp
188
189
190 theorem fullTerminal_fan_two_conductor_bound_of_nonperiodic
191   (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
192   (cap : ℕ → ℕ) (n : ℕ)
193   (shift : (U.forwardIterate cap n).state.Label → ℕ) (K J : ℕ)
194   (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))
195   (m k : ℕ) (hmk : m ≤ k) (X : ℝ) (hX : 0 < X)
196   (hQ : ((3 ^ k : ℕ) : ℝ) ≤ X)
197   (hsource : ∀ z : U.FullTerminalAt cap n shift K,
198     X ≤ (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) ∧
199     (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) < 32 * X)
200   (epsilon : ℝ) (he : 0 ≤ epsilon)
201   (hL1 : ndTernaryUniformMean k (fun y => |ndSyracuseUnitReferenceDensity k y -
202     ndSyracuseUnitReferenceDensity m (Tao.taoZModThreeProjection hmk y)|) ≤ epsilon) :
203   letI := (U.forwardIterate cap n).state.labelFintype
204   (∑ z : U.FullTerminalAt cap n shift K,
205     ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight
206     (U.forwardIterate cap n).state.outerWeight z *
207     ∑ j : Fin J, (1 / 4 : ℝ) ^ j.val * ndSyracuseUnitReferenceDensity k
208     (ndTerminalSourceFan j.val
209       (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z) : ZMod (3 ^ k))) ≤
210     (8 / 9 : ℝ) * ((3 ^ m : ℕ) : ℝ) * U.fullTerminalMass cap n shift K +
211     44 * U.parentSourcePotential * epsilon := by
212   classical
213   letI := (U.forwardIterate cap n).state.labelFintype
214   let W := fun z : U.FullTerminalAt cap n shift K =>
215     ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight
216     (U.forwardIterate cap n).state.outerWeight z
217   let source := fun z : U.FullTerminalAt cap n shift K =>
218     ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z
219   let H := (2 / 3 : ℝ) * ((3 ^ m : ℕ) : ℝ)
220   let delta := fun y : ZMod (3 ^ k) => |ndSyracuseUnitReferenceDensity k y -
221     ndSyracuseUnitReferenceDensity m (Tao.taoZModThreeProjection hmk y)|
222   have hW : ∀ z, 0 ≤ W z := fun z =>
223     ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight_nonneg _
224     (U.forwardIterate cap n).state.weight_nonneg z
225   have hmass : 0 ≤ U.fullTerminalMass cap n shift K := Finset.sum_nonneg fun z _ => hW z
226   have hstep (j : ℕ) :
227     (∑ z, W z * ndSyracuseUnitReferenceDensity k
228       (ndTerminalSourceFan j (source z) : ZMod (3 ^ k))) ≤
229     H * U.fullTerminalMass cap n shift K + 33 * U.parentSourcePotential * epsilon := by
230     let psi := fun y : ZMod (3 ^ k) => delta (ndTerminalResidueFan k j y)
231     have hp : ∀ y, 0 ≤ psi y := fun _ => abs_nonneg _
232     have hmean : ndTernaryUniformMean k psi ≤ epsilon := by
233       rw [show psi = (fun y => delta (ndTerminalResidueFan k j y)) from rfl,
234         terminalResidueFan_fullMean]
235     exact hL1
236   have hc := U.fullTerminal_weighted_residue_census_of_nonperiodic cap n shift K hseed
237   k X hX hQ hsource psi hp
238   have hc' : (∑ z, W z * psi (source z : ZMod (3 ^ k))) ≤

```

```

239      33 * U.parentSourcePotential * epsilon := hc.trans
240      (mul_le_mul_of_nonneg_left hmean (mul_nonneg (by norm_num)
241      ↪ U.frozenSourcePotential_nonneg))
242 have hpoint (x : ℕ) : ndSyracuseUnitReferenceDensity k
243 (ndTerminalSourceFan j x : ZMod (3 ^ k)) ≤ H + psi (x : ZMod (3 ^ k)) := by
244 dsimp [psi, delta]
245 rw [terminalResidueFan_natCast]
246 have hb := unitReferenceDensity_finite_upper m
247 (Tao.taoZModThreeProjection hmk (ndTerminalSourceFan j x : ZMod (3 ^ k)))
248 have ha := le_abs_self (ndSyracuseUnitReferenceDensity k
249 (ndTerminalSourceFan j x : ZMod (3 ^ k)) -
250 ndSyracuseUnitReferenceDensity m
251 (Tao.taoZModThreeProjection hmk (ndTerminalSourceFan j x : ZMod (3 ^ k))))
252 dsimp [H]
253 linarith
254 calc
255 _ ≤ ∑ z, W z * (H + psi (source z : ZMod (3 ^ k))) :=
256   Finset.sum_le_sum fun z _ => mul_le_mul_of_nonneg_left (hpoint _) (hW z)
257 _ = H * U.fullTerminalMass cap n shift K + ∑ z, W z * psi (source z : ZMod (3 ^ k)) :=
258   ↪ by
259     simp_rw [mul_add]
260     rw [Finset.sum_add_distrib, ← Finset.sum_mul]
261     change (∑ z, W z) * H + _ = H * (∑ z, W z) + _
262     ring
263 _ ≤ _ := add_le_add_right hc' _
264 let B := H * U.fullTerminalMass cap n shift K + 33 * U.parentSourcePotential * epsilon
265 have hb : 0 ≤ B := add_nonneg (mul_nonneg (by dsimp [H]; positivity) hmass)
266 (mul_nonneg (mul_nonneg (by norm_num) U.frozenSourcePotential_nonneg) he)
267 calc
268 _ = ∑ j : Fin J, (1 / 4 : ℝ) ^ j.val *
269   ∑ z, W z * ndSyracuseUnitReferenceDensity k
270   (ndTerminalSourceFan j.val (source z) : ZMod (3 ^ k)) := by
271     simp_rw [Finset.mul_sum]
272     rw [Finset.sum_comm]
273     apply Finset.sum_congr rfl
274     intro j _
275     apply Finset.sum_congr rfl
276     intro z _
277     dsimp [W, source]
278     ring
279 _ ≤ ∑ j : Fin J, (1 / 4 : ℝ) ^ j.val * B :=
280   Finset.sum_le_sum fun j _ => mul_le_mul_of_nonneg_left (hstep _) (by positivity)
281 _ = (∑ j : Fin J, (1 / 4 : ℝ) ^ j.val) * B := (Finset.sum_mul _ _).symm
282 _ ≤ (4 / 3 : ℝ) * B := mul_le_mul_of_nonneg_right (terminal_fan_coeff_sum_le J) hb
283 _ = _ := by dsimp [B, H]; ring
284 theorem forwardCoreTerminalUnitMass_le_two_conductor_of_nonperiodic
285 (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
286 (cap width : ℕ → ℕ) (n K m k : ℕ) (hmk : m ≤ k)
287 (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))
288 (X : ℝ) (hX : 0 < X) (hQ : ((3 ^ k : ℕ) : ℝ) ≤ X)
289 (hi : ∀ i : (U.forwardIterate cap n).state.Label,
290   ndGeom2ShiftedWideSymmetricPhysicalIntervalMin (U.forwardIterate cap n).floor
291   ((U.forwardIterate cap n).state.root i) < X ∧
292   X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax (U.forwardIterate cap n).floor
293   ((U.forwardIterate cap n).state.root i))
294 (hsource : ∀ z : (U.forwardIterate cap n).FullTerminalIncidence X hX hi,
295   X ≤ (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) ∧
296   (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) < 32 * X)
297 (epsilon : ℝ) (he : 0 ≤ epsilon)
298 (hL1 : ndTernaryUniformMean k (fun y => |ndSyracuseUnitReferenceDensity k y -
299   ndSyracuseUnitReferenceDensity m (Tao.taoZModThreeProjection hmk y)|) ≤ epsilon) :
300 U.forwardCoreTerminalUnitMass cap width n K k X hX hi ≤
301 (8 / 9 : ℝ) * ((3 ^ m : ℕ) : ℝ) * U.fullTerminalMass cap n

```

```

302      ((U.forwardIterate cap n).fullTerminalShift X hX hi) 1 +
303      44 * U.parentSourcePotential * epsilon := by
304    classical
305    let V := U.forwardIterate cap n
306    let I := V.state.labelFintype
307    have hmask (i : V.state.Label) : U.forwardCoreOuterWeight cap width n i ≤
308      ↪ V.state.outerWeight i := by
309      unfold forwardCoreOuterWeight
310      split_ifs
311      · exact le_rfl
312      · exact V.state.weight_nonneg i
313    apply (U.forwardCoreTerminalUnitMass_le_full_cap_one_fan cap width n K k X hX hi).trans
314    apply le_trans _ (U.fullTerminal_fan_two_conductor_bound_of_nonperiodic cap n
315      ↪ (V.fullTerminalShift X hX hi)
316      1 (K / 2 + 1) hseed m k hmk X hX hQ hsource epsilon he hL1)
317    apply Finset.sum_le_sum
318    intro z _
319    apply mul_le_mul_of_nonneg_right
320    · unfold ndGeom2PredictableRootSideBoundedOvershootIncidenceWeight
321      exact mul_le_mul_of_nonneg_right (hmask _)
322      (ndGeom2PredictableRootSideBoundedOvershootIncidenceAtom_nonneg z)
323    · exact Finset.sum_nonneg fun j _ => mul_nonneg (by positivity)
324      (ndSyracuseUnitReferenceDensity_nonneg _ _)
325
326 324 /-- The physical shell estimate is independent of the eventual target. -/
327 theorem fullTerminal_source_window
328   (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
329   (X : ℝ) (hX : 0 < X)
330   (hi : ∀ i : U.state.Label,
331     ndGeom2ShiftedWideSymmetricPhysicalIntervalMin U.floor (U.state.root i) < X ∧
332     X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax U.floor (U.state.root i))
333   (z : U.FullTerminalIncidence X hX hi) :
334   X ≤ (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) ∧
335   (ndGeom2PredictableRootSideBoundedOvershootIncidenceSource z : ℝ) < 32 * X := by
336   have hpacket : ∀ i, 16 ^ U.floor ≤ U.state.root i := fun i =>
337     (Nat.pow_le_pow_right (by norm_num) (U.floor_le_base i)).trans (U.state.rootLower i)
338   have hs := U.fullTerminalShift_spec X hX hi
339   (ndGeom2PredictableRootSideBoundedOvershootIncidenceLabel z)
340   have hb := ndGeom2RootSideCommonFloor_capOne_source_bounds U.state.root_odd
341   (by have h := U.floor_twoHundred; omega) hpacket z
342   exact ⟨hs.2.1.trans hb.1, by nlinarith only [hb.2, hs.2.2]⟩
343
344 342 theorem forwardCoreTerminalUnitMass_le_native_two_conductor_rate_of_nonperiodic
345   (A : ℕ) (hA : 0 < A) :
346   ∃ Cmix : ℝ, 0 ≤ Cmix ∧
347   ∀ (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
348     (cap width : ℕ → ℕ) (n K m k : ℕ),
349     1 ≤ m → (hmk : m ≤ k) → k ≤ (U.forwardIterate cap n).floor →
350     (∀ i, Nonperiodic Tao.syracuse (U.state.root i)) →
351     ∀ (X : ℝ) (hX : 0 < X)
352     (hi : ∀ i : (U.forwardIterate cap n).state.Label,
353       ndGeom2ShiftedWideSymmetricPhysicalIntervalMin (U.forwardIterate cap n).floor
354       ((U.forwardIterate cap n).state.root i) < X ∧
355       X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax (U.forwardIterate cap n).floor
356       ((U.forwardIterate cap n).state.root i)),
357     0 < U.forwardCoreTerminalUnitMass cap width n K k X hX hi →
358     U.forwardCoreTerminalUnitMass cap width n K k X hX hi ≤
359     (8 / 9 : ℝ) * ((3 ^ m : ℕ) : ℝ) * U.fullTerminalMass cap n
360     ((U.forwardIterate cap n).fullTerminalShift X hX hi) 1 +
361     44 * U.parentSourcePotential * (Cmix / (m : ℝ) ^ A) := by
362   obtain ⟨Cmix, hCmix, hmix⟩ := unitReferenceDensity_fulll1_polynomial_rate A hA
363   refine ⟨Cmix, hCmix, ?_⟩
364   intro U cap width n K m k hm hmk hk hseed X hX hi hT
365   let V := U.forwardIterate cap n
366   have hQ := U.terminal_conductor_room_of_pos_mark cap width n K k hk X hX hi hT

```

```

365 exact U.forwardCoreTerminalUnitMass_le_two_conductor_of_nonperiodic cap width n K m k hmk
    ↪ hseed
366 X hX hQ.le hi
367 (fun z => V.fullTerminal_source_window X hX hi z)
368 (Cmix / (m : ℝ) ^ A) (div_nonneg hCmix (by positivity)) (hmix m k hm hmk)
369
370
371 end NDEdge2ShiftedWideSymmetricRootSideUniformFloorState
372 end
373 end Erdos1135.ND.PositiveDensity

```

B.6 CollatzTargetDensity/Activation.lean

```

1 import CollatzTargetDensity.SeedSupply
2 import Erdos1135.ND.PositiveDensity.GeoM2ShiftedWideSymmetricTerminalShiftImageRate
3
4 /-!+# Target-independent activation from residue-wise nonperiodic seeds
5
6 Adapted from Lech Mazur's Apache-2.0 positive-density package, corrected commit
7 0aef8b0cfaaf6959500063472f2e328c60df8d54. The original uniform analytic estimates
8 are imported unchanged. Only the physical seed-selection step is generalized.
9 -/
10
11 namespace CollatzTargetDensity
12 open Erdos1135 Erdos1135.ND.PositiveDensity
13 open Filter
14 open scoped Topology
15 noncomputable section
16
17 theorem seed_at_generation {target : ℕ} (hsupply : SeedSupply target)
18   {b k : ℕ} (hb : 200 ≤ b) (hk : 1 ≤ k) (cap width : ℕ → ℕ)
19   (hwidth : ∀ B, b ≤ B → 2 ≤ width B) (n X : ℕ) :
20   ∃ (M : ℕ) (hM : Odd M) (hl : 16 ^ b ≤ M),
21     X < M ∧ Tao.syracuse M = target ∧ Nonperiodic Tao.syracuse M ∧
22     (2 / 3 : ℝ) * ndRootCoreProbabilityProduct b cap width n ≤
23     (ndRootCoreSingletonState b M hb hM hl).forwardCoreMarkedMass cap width n k (fun _ =>
24       ↪ 1) := by
25   obtain ⟨y, _, hy⟩ := exists_unit_rootCoreBackwardMark_ge_product
26     (by omega : 9 ≤ b) hk cap width hwidth n
27   let q := ndRootCoreBackwardConductor b k n
28   let r : Fin (3 ^ q) := ⟨y.val, ZMod.val_lt y⟩
29   obtain ⟨M, hM, hlarge, hhit, hres, hnonperiodic⟩ := hsupply q (max X (16 ^ b)) r
30   have hl : 16 ^ b ≤ M := (le_max_right _ _).trans hlarge.le
31   have heq : (M : ZMod (3 ^ q)) = y := by
32     calc
33     _ = ((M % 3 ^ q : ℕ) : ZMod (3 ^ q)) := by simp
34     _ = ((y.val : ℕ) : ZMod (3 ^ q)) := congrArg (fun a : ℕ => (a : ZMod (3 ^ q))) hres
35     _ = y := ZMod.natCast_zmod_val y
36   refine ⟨M, hM, hl, (le_max_left _ _).trans_lt hlarge, hhit, hnonperiodic, ?_⟩
37   rw [rootCoreSingletonState_markedMass_eq_backwardMark _ _ _ _ _ _ _ _ _ _ hk]
38   simpa only [heq] using hy
39
40 theorem positive_core_limit {target : ℕ} (hsupply : SeedSupply target)
41   {b : ℕ} (hb : 32 ^ 5 ≤ b) (K0 X : ℕ) :
42   ∃ (M : ℕ) (hM : Odd M) (hl : 16 ^ b ≤ M),
43     X < M ∧ Tao.syracuse M = target ∧ Nonperiodic Tao.syracuse M ∧ ∃ ell : ℝ, 0 < ell ∧
44     Tendsto ((ndRootCoreSingletonState b M (by omega) hM hl).coreMarkedSequence
45       (fun n => K0 + n / 100) ndRootCoreWidth) atTop (ℳ ell) := by
46   let cap (n : ℕ) := K0 + n / 100
47   have hb200 : 200 ≤ b := by omega
48   have hwidth (B : ℕ) (hB : b ≤ B) : 2 ≤ ndRootCoreWidth B := by
49     have h := (rootCoreWidth_guard (hb.trans hB)).1
50     omega
51   have hupper (n : ℕ) : cap n ≤ (K0 + 1) * (n + 1) := by
52     have h := Nat.div_le_self n 100

```

```

52   dsimp [cap]
53   nlinarith
54   have hlower (n : ℕ) : K0 + n / 100 ≤ cap n := le_rfl
55   obtain ⟨M0, hM0, hL0, _, _, _⟩ :=
56     seed_at_generation hsupply hb200 (by norm_num : 1 ≤ 1)
57     cap ndRootCoreWidth hwidth 0 0
58   let U0 := ndRootCoreSingletonState b M0 hb200 hM0 hL0
59   obtain ⟨p, hp, hprod⟩ := U0.exists_pos_le_coreProbabilityProduct hb cap K0 hlower
60   change ∀ n, p ≤ ndRootCoreProbabilityProduct b cap ndRootCoreWidth n at hprod
61   obtain ⟨C, hC, hlimall⟩ :=
62     NDGeom2ShiftedWideSymmetricRootSideUniformFloorState.exists_coreMarkedLimit_with_root_uni
63     ↪ form_tail
64   have ht : ∀f N in atTop, ndRootCoreVariationTail b (K0 + 1) K0 C N < p / 3 :=
65     (tendsto_order.mp (tendsto_rootCoreVariationTail_zero b (K0 + 1) K0 C)).2 (p / 3) (by
66       ↪ positivity)
67   obtain ⟨N, hN⟩ := Filter.eventually_atTop.mp ht
68   let k := (U0.forwardIterate cap N).floor / 4
69   have hk : 1 ≤ k := by have h := (U0.forwardIterate cap N).floor_twoHundred; dsimp [k]; omega
70   obtain ⟨M, hM, hL, hX, hhit, hnonperiodic, hmark⟩ :=
71     seed_at_generation hsupply hb200 hk cap ndRootCoreWidth hwidth N X
72   let U := ndRootCoreSingletonState b M hb200 hM hL
73   have hfloor := U.forward_floor_eq_of_floor_eq U0 cap cap rfl N
74   have hmarked : (2 / 3 : ℝ) * ndRootCoreProbabilityProduct b cap ndRootCoreWidth N ≤
75     U.coreMarkedSequence cap ndRootCoreWidth N := by
76     unfold NDGeom2ShiftedWideSymmetricRootSideUniformFloorState.coreMarkedSequence
77     rw [hfloor]
78     exact hmark
79   obtain ⟨ell, _, hlim, htail⟩ := hlimall U hb cap (K0 + 1) K0 hupper hlower
80   have hdev := (abs_le.mp (htail N)).1
81   have hD : U.denominator = 1 := rootCoreSingletonState_denominator_eq_one b M hb200 hM hL
82   rw [hD, one_mul] at hdev
83   change -(ndRootCoreVariationTail b (K0 + 1) K0 C N) ≤
84     ell - U.coreMarkedSequence cap ndRootCoreWidth N at hdev
85   refine ⟨M, hM, hL, hX, hhit, hnonperiodic, ell, ?_, hlim⟩
86   have hn := hN N le_rfl
87   have hpn := hprod N
88   nlinarith
89   theorem uniform_terminal_mark {target : ℕ} (hsupply : SeedSupply target)
90     {b : ℕ} (hb : 32 ^ 5 ≤ b) (K0 R : ℕ) :
91     ∃ (M : ℕ) (hM : Odd M) (hL : 16 ^ b ≤ M),
92     R < M ∧ Tao.syracuse M = target ∧ Nonperiodic Tao.syracuse M ∧ ∃ a : ℝ, 0 < a ∧
93     ∀f n in atTop,
94     let U := ndRootCoreSingletonState b M (by omega) hM hL;
95     let cap := fun n => K0 + n / 100;
96     let V := U.forwardIterate cap n;
97     ∀ (X : ℝ) (hX : 0 < X)
98     (hi : ∀ i : V.state.Label,
99       ndGeom2ShiftedWideSymmetricPhysicalIntervalMin V.floor (V.state.root i) < X ∧
100       X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax V.floor (V.state.root i)),
101     a ≤ U.forwardCoreTerminalUnitMass cap ndRootCoreWidth n (cap n) (V.floor / 4) X hX hi
102     ↪ := by
103   obtain ⟨M, hM, hL, hR, hhit, hnonperiodic, ell, hell, hlim⟩ :=
104     positive_core_limit hsupply hb K0 R
105   let U := ndRootCoreSingletonState b M (by omega) hM hL
106   have hspan : U.rootSpan 0 := by intro i j; change M ≤ 2 ^ 0 * M; simp
107   have hupper (n : ℕ) : K0 + n / 100 ≤ (K0 + 1) * (n + 1) := by
108     have h := Nat.div_le_self n 100
109     nlinarith
110   exact ⟨M, hM, hL, hR, hhit, hnonperiodic, ell / 2, by positivity,
111     U.eventually_terminalUnitMass_ge_of_pos_core_limit _ 0 (K0 + 1) K0 hspan hupper (fun _ =>
112       ↪ le_rfl) hell hlim⟩
113 end

```

```
113 end CollatzTargetDensity
```

B.7 CollatzTargetDensity/RawBridge.lean

```
1 import CollatzTargetDensity.SeedSupply
2 import Erdos1135.Tao.Syracuse.CollatzBridge
3 import Erdos1135.Parity
4
5 /-! Deterministic reachability and transfer to even targets. -/
6 namespace CollatzTargetDensity
7 open Erdos1135
8 noncomputable section
9
10 def oddReaches (target : ℕ) : Set ℕ :=
11   {n | Odd n ∧ ∃ k : ℕ, (Tao.syracuse^[k]) n = target}
12
13 /-- Choose an odd predecessor prime to 3 of any target prime to 3.
14   This construction also works when the target is even. -/
15 theorem exists_fertile_raw_child {target : ℕ} (hthree : target % 3 ≠ 0) :
16   ∃ c : ℕ, Odd c ∧ c % 3 ≠ 0 ∧ Reaches c target := by
17   have hbase : ∃ c v : ℕ, Odd c ∧ 3 * c + 1 = 2 ^ v * target := by
18     by_cases h : target % 3 = 1
19     · let c := (4 * target - 1) / 3
20       have he : 3 * c + 1 = 4 * target := by dsimp [c]; omega
21       exact ⟨c, 2, ⟨c / 2, by omega⟩, he⟩
22     · let c := (2 * target - 1) / 3
23       have he : 3 * c + 1 = 2 * target := by dsimp [c]; omega
24       exact ⟨c, 1, ⟨c / 2, by omega⟩, he⟩
25   have hfertile : ∃ c v : ℕ, Odd c ∧ c % 3 ≠ 0 ∧ 3 * c + 1 = 2 ^ v * target := by
26     obtain ⟨c, v, hc, he⟩ := hbase
27     by_cases h : c % 3 ≠ 0
28     · exact ⟨c, v, hc, h, he⟩
29     · refine ⟨4 * c + 1, v + 2, ⟨2 * c, by omega⟩, by omega, ?_⟩
30       calc
31         _ = 4 * (3 * c + 1) := by ring
32         _ = 4 * (2 ^ v * target) := by rw [he]
33         _ = _ := by rw [pow_add]; ring
34   obtain ⟨c, v, hc, hunit, he⟩ := hfertile
35   refine ⟨c, hc, hunit, v + 1, ?_⟩
36   rw [Function.iterate_add_apply, Function.iterate_one,
37     collatzStep_eq_three_mul_add_one_of_odd hc, he]
38   exact Tao.collatz_iterate_powTwo_mul v target
39
40 def rawReaches (target : ℕ) : Set ℕ :=
41   {n | 0 < n ∧ Reaches n target}
42
43 theorem oddReaches_subset_rawReaches {seed target : ℕ} (hseed : Reaches seed target) :
44   oddReaches seed ⊆ rawReaches target := by
45   intro n hn
46   obtain ⟨hnodd, k, hk⟩ := hn
47   have hreach : Reaches n seed := by
48     simpa only [hk] using Tao.reaches_syracuse_iterate hnodd k
49   exact ⟨hnodd.pos, hreach.trans hseed⟩
50
51 end
52 end CollatzTargetDensity
```

B.8 CollatzTargetDensity/Assembly.lean

```
1 import CollatzTargetDensity.SourceCharge
2 import CollatzTargetDensity.Activation
3 import CollatzTargetDensity.RawBridge
4 import Erdos1135.ND.PositiveDensity.Geom2ShiftedWideSymmetricPositiveDensityAssembly
5
6 /-!
```

```

7 # Density assembly with an arbitrary target
8
9 Adapted from Lech Mazur's Apache-2.0 source, corrected commit
10 0aef8b0cfaaf6959500063472f2e328c60df8d54. The analytic inputs and interval
11 synchronization are unchanged. The conclusion records a supplied target;
12 nonperiodicity replaces seed convergence in the counting step.
13 -/
14 namespace Erdos1135.ND.PositiveDensity
15 open scoped BigOperators
16 open Filter CollatzTargetDensity
17 noncomputable section
18 namespace NDGeom2ShiftedWideSymmetricRootSideUniformFloorState
19
20 theorem eventually_pos_fullTerminalMass_of_nonperiodic_mark
21   (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
22   (cap width : ℕ → ℕ) {a : ℝ}
23   (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))
24   (ha : 0 < a)
25   (hmark : ∀f n in atTop,
26     let V := U.forwardIterate cap n;
27     ∀ (X : ℝ) (hX : 0 < X)
28       (hi : ∀ i : V.state.Label,
29         ndGeom2ShiftedWideSymmetricPhysicalIntervalMin V.floor (V.state.root i) < X ∧
30         X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax V.floor (V.state.root i)),
31     a ≤ U.forwardCoreTerminalUnitMass cap width n (cap n) (V.floor / 4) X hX hi) :
32   ∃ eta : ℝ, 0 < eta ∧ ∀f n in atTop,
33     let V := U.forwardIterate cap n;
34     ∀ (X : ℝ) (hX : 0 < X)
35       (hi : ∀ i : V.state.Label,
36         ndGeom2ShiftedWideSymmetricPhysicalIntervalMin V.floor (V.state.root i) < X ∧
37         X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax V.floor (V.state.root i)),
38     eta ≤ U.fullTerminalMass cap n (V.fullTerminalShift X hX hi) 1 := by
39   obtain ⟨Cmix, hCmix, hrate⟩ :=
40     ↪ forwardCoreTerminalUnitMass_le_native_two_conductor_rate_of_nonperiodic 2 (by decide)
41   obtain ⟨m, hm⟩ := exists_nat_gt (max 1 (88 * U.parentSourcePotential * Cmix / a))
42   have hm1 : (1 : ℝ) < m := (le_max_left _).trans_lt hm
43   have hmNat : 1 ≤ m := by exact_mod_cast hm1.le
44   have hmPos : (0 : ℝ) < m := by linarith
45   have hcoarse : 44 * U.parentSourcePotential * (Cmix / (m : ℝ) ^ 2) ≤ a / 2 := by
46     have hmBig : 88 * U.parentSourcePotential * Cmix / a < (m : ℝ) :=
47       (le_max_right _).trans_lt hm
48     have hmProd := (div_lt_iff₀ ha).1 hmBig
49     have hmSq : (m : ℝ) ≤ (m : ℝ) ^ 2 := by nlinarith
50     have hpay := mul_le_mul_of_nonneg_left hmSq ha.le
51     rw [← mul_div_assoc]
52     apply (div_le_iff₀ (by positivity : (0 : ℝ) < (m : ℝ) ^ 2)).2
53     nlinarith
54   let H := (8 / 9 : ℝ) * ((3 ^ m : ℕ) : ℝ)
55   have hH : 0 < H := by dsimp [H]; positivity
56   refine ⟨a / (2 * H), by positivity, ?_⟩
57   filter_upwards [hmark, eventually_ge_atTop (2 * m)] with n hn hnlarge
58   intro V X hX hi
59   have hf : U.floor + 2 * n ≤ V.floor := by
60     have h := U.floor_add_two_mul_le_geometricIntervalStoppedIterate_floor cap 0 n
61     rw [U.geometricIntervalStoppedIterate_floor_eq_iterate cap 0 n] at h
62     simpa only [V, U.forwardIterate_eq_iterate cap n] using h
63   have hmk : m ≤ V.floor / 4 := by omega
64   have hT := hn X hX hi
65   have h := hrate U cap width n (cap n) m (V.floor / 4) hmNat hmk
66   (Nat.div_le_self _ _) hseed X hX hi (ha.trans_le hT)
67   apply (div_le_iff₀ (mul_pos (by norm_num) hH)).2
68   change a ≤ U.fullTerminalMass cap n (V.fullTerminalShift X hX hi) 1 * (2 * H)
69   dsimp [H]
70   nlinarith [hcoarse]

```

```

71
72 theorem targetDensity_of_eventually_pos_fullTerminalMass
73   (U : NDGeom2ShiftedWideSymmetricRootSideUniformFloorState)
74   (cap : ℕ → ℕ) {e L : ℕ} (he : U.rootSpan e)
75   (hcap : ∀ n, cap n ≤ L * (n + 1))
76   {eta : ℝ} (target : ℕ)
77   (hseed : ∀ i, Nonperiodic Tao.syracuse (U.state.root i))
78   (htarget : ∀ i, ∃ k : ℕ, (Tao.syracuse^[k]) (U.state.root i) = target)
79   (hP : 0 < U.parentSourcePotential) (heta : 0 < eta)
80   (hmass : ∀f n in atTop,
81     let V := U.forwardIterate cap n;
82     ∀ (X : ℝ) (hX : 0 < X)
83       (hi : ∀ i : V.state.Label,
84         ndGeom2ShiftedWideSymmetricPhysicalIntervalMin V.floor (V.state.root i) < X ∧
85         X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax V.floor (V.state.root i)),
86     eta ≤ U.fullTerminalMass cap n (V.fullTerminalShift X hX hi) 1) :
87   ∀f Y : ℕ in atTop,
88     (eta / (32 * U.parentSourcePotential)) * (Y : ℝ) ≤
89     (Terras.natCount (oddReaches target) Y : ℝ) := by
90   classical
91   obtain ⟨N0, hN0⟩ := eventually_atTop.mp hmass
92   let Nstar := max N0 (1000000000 * (e + L + 3))
93   let V0 := U.iterate cap Nstar
94   let I := V0.state.labelFintype
95   let X0 := ∑ i : V0.state.Label,
96     max 0 (ndGeom2ShiftedWideSymmetricPhysicalIntervalMax V0.floor (V0.state.root i))
97   have hstart (X : ℝ) (hXX : X0 < X) : ∀ i : V0.state.Label,
98     ndGeom2ShiftedWideSymmetricPhysicalIntervalMax V0.floor (V0.state.root i) < X := by
99     intro i
100     have hsingle := Finset.single_le_sum
101       (f := fun j : V0.state.Label => max 0
102         (ndGeom2ShiftedWideSymmetricPhysicalIntervalMax V0.floor (V0.state.root j)))
103       (fun j _ => le_max_left _ _) (Finset.mem_univ i)
104     exact ((le_max_right _ _).trans hsingle).trans_lt hXX
105   apply eventually_atTop.mpr
106   refine ⟨Nat.ceil (32 * (max X0 0 + 1)), ?_⟩
107   intro Y hY
108   have hYlarge : 32 * (max X0 0 + 1) ≤ (Y : ℝ) :=
109     (Nat.le_ceil _).trans (by exact_mod_cast hY)
110   let X := (Y : ℝ) / 32
111   have hX : 0 < X := by dsimp [X]; have := le_max_right X0 0; linarith
112   have hXX : X0 < X := by dsimp [X]; have := le_max_left X0 0; linarith
113   obtain ⟨n, hn, hi⟩ := U.exists_unstopped_generation_all_intervals_contain he cap hcap
114     Nstar (le_max_right _ _) X (hstart X hXX)
115   have hif : ∀ i : (U.forwardIterate cap n).state.Label,
116     ndGeom2ShiftedWideSymmetricPhysicalIntervalMin (U.forwardIterate cap n).floor
117       ((U.forwardIterate cap n).state.root i) < X ∧
118     X ≤ ndGeom2ShiftedWideSymmetricPhysicalIntervalMax (U.forwardIterate cap n).floor
119       ((U.forwardIterate cap n).state.root i) := by
120     rw [U.forwardIterate_eq_iterate cap n]
121     exact hi
122   let shift := (U.forwardIterate cap n).fullTerminalShift X hX hif
123   let sources := U.fullTerminalSources cap n shift 1
124   have hm : eta ≤ U.fullTerminalMass cap n shift 1 :=
125     hN0 n ((le_max_left _ _).trans hn) X hX hif
126   have hwindow (z : U.FullTerminalAt cap n shift 1) :=
127     (U.forwardIterate cap n).fullTerminal_source_window X hX hif z
128   have hcountMass := U.fullTerminal_mass_mul_lower_le_frozenPotential_mul_card_of_nonperiodic
129     cap n shift 1 hseed X (fun z => (hwindow z).1)
130   have hmember (source : ℕ) (hs : source ∈ sources) :
131     source ∈ oddReaches target ∧ X ≤ (source : ℝ) ∧ (source : ℝ) < 32 * X := by
132     obtain ⟨z, _, rfl⟩ := Finset.mem_image.mp hs
133     let p := U.fullTerminalPath cap n shift 1 z
134     obtain ⟨k, hk⟩ := htarget (fullTerminalAncestor z)
135     refine ⟨p.sourceOdd, k + p.depth, ?_⟩, hwindow z

```

```

136   rw [Function.iterate_add_apply, p.terminal_eq, hk]
137   have hrange : sources  $\subseteq$  Finset.range Y := by
138     intro source hs
139     have h := (hmember source hs).2.2
140     have hlt : (source :  $\mathbb{R}$ ) < Y := by dsimp [X] at h; linarith
141     exact Finset.mem_range.mpr (by exact_mod_cast hlt)
142   have htargetS : ( $\uparrow$ sources : Set  $\mathbb{N}$ )  $\subseteq$  oddReaches target :=
143     fun source hs => (hmember source hs).1
144   have hcard : sources.card  $\leq$  Terras.natCount (oddReaches target) Y :=
145     finset_card_le_natCount_of_subset_range hrange htargetS
146   have hcount : X * U.fullTerminalMass cap n shift 1  $\leq$ 
147     U.parentSourcePotential * (Terras.natCount (oddReaches target) Y :  $\mathbb{R}$ ) :=
148     hcountMass.trans (mul_le_mul_of_nonneg_left (by exact_mod_cast hcard) hP.le)
149   have htotal := (mul_le_mul_of_nonneg_left hm hX.le).trans hcount
150   rw [div_mul_eq_mul_div]
151   apply (div_le_iff₀ (by positivity : (0 :  $\mathbb{R}$ ) < 32 * U.parentSourcePotential)).2
152   dsimp [X] at htotal
153   nlinarith only [htotal]
154
155
156 end NDGeom2ShiftedWideSymmetricRootSideUniformFloorState
157 end
158 end Erdos1135.ND.PositiveDensity
159
160 namespace CollatzTargetDensity
161 open Erdos1135 Erdos1135.ND.PositiveDensity Filter
162 noncomputable section
163
164 /-- Analytic conclusion from residue-wise nonperiodic seed supply.
165   The arithmetic supply is an explicit input, not an assumed Collatz result. -/
166 theorem positive_density_of_seed_supply {target :  $\mathbb{N}$ } (hsupply : SeedSupply target) :
167    $\exists c : \mathbb{R}, 0 < c \wedge \forall^f X : \mathbb{N}$  in atTop,
168      $c * (X : \mathbb{R}) \leq$  (Terras.natCount (oddReaches target) X :  $\mathbb{R}$ ) := by
169   classical
170   obtain ⟨M, hM, hL, hMpos, hhit, hnonperiodic, a, ha, hmark⟩ :=
171     uniform_terminal_mark hsupply (b := 32 ^ 5) (by rfl) 0 0
172   let U := ndRootCoreSingletonState (32 ^ 5) M (by norm_num) hM hL
173   let cap := fun n :  $\mathbb{N}$  => 0 + n / 100
174   have hseed :  $\forall i$ , Nonperiodic Tao.syracuse (U.state.root i) := fun _ => hnonperiodic
175   have htarget :  $\forall i, \exists k : \mathbb{N},$  (Tao.syracuse^[k]) (U.state.root i) = target := by
176     intro i
177     exact ⟨1, hhit⟩
178   have hspan : U.rootSpan 0 := by intro i j; change M  $\leq$  2 ^ 0 * M; simp
179   have hcap :  $\forall n, \text{cap } n \leq 1 * (n + 1)$  := by intro n; dsimp [cap]; omega
180   have hP : 0 < U.parentSourcePotential := by
181     change 0 < ndGeom2PredictableRootSideParentSourcePotential (Finset.univ : Finset Unit)
182     (fun _ => (1 :  $\mathbb{R}$ )) (fun _ => M)
183     simpa [ndGeom2PredictableRootSideParentSourcePotential] using
184       (show (0 :  $\mathbb{R}$ ) < M by exact_mod_cast hMpos)
185   obtain ⟨eta, heta, hmass⟩ := U.eventually_pos_fullTerminalMass_of_nonperiodic_mark
186     cap ndRootCoreWidth hseed ha hmark
187   exact ⟨eta / (32 * U.parentSourcePotential), by positivity,
188     U.targetDensity_of_eventually_pos_fullTerminalMass cap hspan hcap
189     target hseed htarget hP heta hmass⟩
190
191 end
192 end CollatzTargetDensity

```

B.9 CollatzTargetDensity/AllTargets.lean

```

1 import CollatzTargetDensity.ArithmeticSeeds
2 import CollatzTargetDensity.Assembly
3 import CollatzTargetDensity.RawBridge
4
5 /-!+# Proposed all-target predecessor density endpoint

```

```

6
7 This file must pass the complete build and an axiom audit before its theorem
8 is reported as verified. The new argument depends on the imported analytic
9 machinery of Lech Mazur's positive-density theorem.
10 -/
11
12 namespace CollatzTargetDensity
13 open Erdos1135 Filter
14 noncomputable section
15
16 theorem positive_density_odd_target {target : ℕ}
17   (hodd : Odd target) (hthree : target % 3 ≠ 0) :
18   ∃ c : ℝ, 0 < c ∧ ∀f X : ℕ in atTop,
19   c * (X : ℝ) ≤ (Terras.natCount (oddReaches target) X : ℝ) :=
20   positive_density_of_seed_supply (seed_supply hodd hthree)
21
22 /-- Every positive target prime to 3 has a predecessor set of positive lower
23   natural density. This is not a statement that every integer reaches 1. -/
24 theorem positive_density_all_targets {target : ℕ} (hthree : target % 3 ≠ 0) :
25   ∃ c : ℝ, 0 < c ∧ ∃ X₀ : ℕ, ∀ X : ℕ, X₀ ≤ X →
26   c * (X : ℝ) ≤ (Terras.natCount (rawReaches target) X : ℝ) := by
27   obtain ⟨seed, hodd, hunit, hhit⟩ := exists_fertile_raw_child hthree
28   obtain ⟨c, hc, hcount⟩ := positive_density_odd_target hodd hunit
29   obtain ⟨X₀, hX₀⟩ := eventually_atTop.mp hcount
30   refine ⟨c, hc, X₀, ?_⟩
31   intro X hX
32   have hmono := Terras.natCount_mono (oddReaches_subset_rawReaches hhit) X
33   exact (hX₀ X hX).trans (by exact_mod_cast hmono)
34
35 end
36 end CollatzTargetDensity

```

B.10 CollatzTargetDensity/ThreeDivisible.lean

```

1 import CollatzTargetDensity.RawBridge
2 import Erdos1135.Terras.Density.NaturalDensity
3
4 /-! Exact predecessor structure for the excluded targets divisible by 3. -/
5 namespace CollatzTargetDensity
6 open Erdos1135
7
8 theorem reaches_three_divisible_iff {target n : ℕ} (hthree : target % 3 = 0) :
9   Reaches n target ↔ ∃ k : ℕ, n = 2 ^ k * target := by
10   constructor
11   · rintro ⟨m, hm⟩
12     induction m generalizing n with
13     | zero => exact ⟨0, by simp using hm⟩
14     | succ m ih =>
15       rw [Function.iterate_succ_apply] at hm
16       obtain ⟨k, hk⟩ := ih hm
17       have hstep : collatzStep n % 3 = 0 := by
18         rw [hk, Nat.mul_mod, hthree]
19         simp
20       have he : Even n := by
21         by_contra hne
22         rw [collatzStep_eq_three_mul_add_one_of_not_even hne] at hstep
23         omega
24       have hmod := Nat.even_iff.mp he
25       refine ⟨k + 1, ?_⟩
26       calc
27         n = 2 * (n / 2) := by omega
28         _ = 2 * collatzStep n := by rw [collatzStep_eq_div_two_of_even he]
29         _ = 2 * (2 ^ k * target) := by rw [hk]
30         _ = 2 ^ (k + 1) * target := by rw [pow_succ]; ring
31   · rintro ⟨k, rfl⟩

```

```

32   exact Tao.reaches_powTwo_mul k target
33
34 theorem three_divisible_count_at_dyadic_cutoff {target : ℕ}
35   (hpos : 0 < target) (hthre : target % 3 = 0) (k : ℕ) :
36   Terras.natCount (rawReaches target) (2 ^ k * target) = k := by
37   classical
38   have hmono : StrictMono (fun j : ℕ => 2 ^ j * target) := by
39     intro i j hij
40     exact Nat.mul_lt_mul_of_pos_right
41       ((Nat.pow_lt_pow_iff_right (by decide : 1 < (2 : ℕ))).2 hij) hpos
42   have hfilter : (Finset.range (2 ^ k * target)).filter (fun n => n ∈ rawReaches target) =
43     (Finset.range k).image (fun j => 2 ^ j * target) := by
44     ext n
45     simp only [Finset.mem_filter, Finset.mem_range, Finset.mem_image]
46     constructor
47     · rintro ⟨hnlt, hn⟩
48       obtain ⟨j, hj⟩ := (reaches_three_divisible_iff hthre).1 hn.2
49       refine ⟨j, ?_, hj.symm⟩
50       rw [hj] at hnlt
51       exact hmono.lt_iff_lt.mp hnlt
52     · rintro ⟨j, hj, rfl⟩
53       exact ⟨hmono hj, Nat.mul_pos (by positivity) hpos, Tao.reaches_powTwo_mul j target⟩
54   unfold Terras.natCount
55   rw [Nat.count_eq_card_filter_range, hfilter,
56     Finset.card_image_of_injective _ hmono.injective, Finset.card_range]
57
58 theorem square_le_two_pow {k : ℕ} (hk : 4 ≤ k) : k ^ 2 ≤ 2 ^ k := by
59   induction k, hk using Nat.le_induction with
60   | base => decide
61   | succ k hk ih =>
62     rw [pow_succ (2 : ℕ) k]
63     nlinarith
64
65 theorem no_positive_lower_density_three_divisible {target : ℕ}
66   (hpos : 0 < target) (hthre : target % 3 = 0) :
67   ¬∃ c : ℝ, 0 < c ∧ ∃ X₀ : ℕ, ∀ X : ℕ, X₀ ≤ X →
68     c * (X : ℝ) ≤ (Terras.natCount (rawReaches target) X : ℝ) := by
69   rintro ⟨c, hc, X₀, hX₀⟩
70   obtain ⟨k, hk⟩ := exists_nat_gt (max ((max X₀ 4 : ℕ) : ℝ) (1 / c))
71   have hknat : max X₀ 4 < k := by exact_mod_cast (le_max_left _).trans_lt hk
72   have hk4 : 4 ≤ k := by omega
73   have hck : 1 < (k : ℝ) * c :=
74     (div_lt_iff₀ hc).1 ((le_max_right _).trans_lt hk)
75   have hsquare : k ^ 2 ≤ 2 ^ k * target :=
76     (square_le_two_pow hk4).trans (Nat.le_mul_of_pos_right _ hpos)
77   have hroom : X₀ ≤ 2 ^ k * target := by
78     have hk_le_square : k ≤ k ^ 2 := by nlinarith
79     omega
80   have hcount := hX₀ (2 ^ k * target) hroom
81   rw [three_divisible_count_at_dyadic_cutoff hpos hthre k] at hcount
82   have hsquare_real : (k : ℝ) ^ 2 ≤ ((2 ^ k * target : ℕ) : ℝ) := by
83     exact_mod_cast hsquare
84   have hpaid := mul_le_mul_of_nonneg_left hsquare_real hc.le
85   have hkpos : (0 : ℝ) < k := by exact_mod_cast (show 0 < k by omega)
86   nlinarith
87
88 end CollatzTargetDensity

```

B.11 CollatzTargetDensity/Public.lean

```

1 import CollatzTargetDensity.AllTargets
2 import CollatzTargetDensity.ThreeDivisible
3
4 /-! Literal map and count interface for the all-target density extension. -/
5 namespace CollatzTargetDensity

```

```

6
7 def step (n : ℕ) : ℕ := if Even n then n / 2 else 3 * n + 1
8
9 def reachesTarget (target n : ℕ) : Prop :=
10   0 < n ∧ ∃ k : ℕ, (step^[k]) n = target
11
12 open Classical in
13 noncomputable def countBelow (target X : ℕ) : ℕ :=
14   Nat.count (reachesTarget target) X
15
16 /-- The count uses positive starting integers strictly below X and the
17   ordinary Collatz map. The lower-density constant may depend on target. -/
18 theorem positive_density (target : ℕ) (hpos : 0 < target) (hthre : target % 3 ≠ 0) :
19   ∃ c : ℝ, 0 < c ∧ ∃ X₀ : ℕ, ∀ X : ℕ, X₀ ≤ X →
20     c * (X : ℝ) ≤ (countBelow target X : ℝ) := by
21   exact positive_density_all_targets hthre
22
23 theorem positive_density_iff (target : ℕ) (hpos : 0 < target) :
24   (∃ c : ℝ, 0 < c ∧ ∃ X₀ : ℕ, ∀ X : ℕ, X₀ ≤ X →
25     c * (X : ℝ) ≤ (countBelow target X : ℝ)) ↔ target % 3 ≠ 0 := by
26   constructor
27   · intro h hthre
28     exact no_positive_lower_density_three_divisible hpos hthre h
29   · exact positive_density target hpos
30
31 end CollatzTargetDensity

```

B.12 CollatzTargetDensity/OrbitInvariance.lean

```

1 import CollatzTargetDensity.RawBridge
2
3 /-! Elementary orbit facts for interpreting the predecessor density result. -/
4 namespace CollatzTargetDensity
5 open Erdos1135
6
7 theorem reaches_one_forward {n a : ℕ} (hna : Reaches n a) (hn : Reaches n 1) :
8   Reaches a 1 := by
9   obtain ⟨d, hd⟩ := hna
10  obtain ⟨t, ht⟩ := hn
11  have hcycle : (collatzStep^[3]) 1 = 1 := by decide
12  have hperiod : (collatzStep^[3 * d]) 1 = 1 := by
13    simpa only [Nat.mul_comm] using periodic_multiples hcycle d
14  refine ⟨t + 2 * d, ?_⟩
15  rw [← hd, ← Function.iterate_add_apply,
16     show t + 2 * d + d = 3 * d + t by omega,
17     Function.iterate_add_apply, ht]
18  exact hperiod
19
20 theorem reaches_three_unit (n : ℕ) (hn : 0 < n) :
21   ∃ a : ℕ, a % 3 ≠ 0 ∧ Reaches n a := by
22   induction n using Nat.strong_induction_on with
23   | h n ih =>
24     by_cases hthre : n % 3 ≠ 0
25     · exact ⟨n, hthre, reaches_refl n⟩
26     · by_cases he : Even n
27       · have hmod := Nat.even_iff.mp he
28         have hpos : 0 < n / 2 := by omega
29         have hlt : n / 2 < n := Nat.div_lt_self hn (by decide)
30         obtain ⟨a, ha, hreach⟩ := ih (n / 2) hlt hpos
31         refine ⟨a, ha, reaches_of_reaches_step ?_⟩
32         simpa only [collatzStep_eq_div_two_of_even he] using hreach
33       · exact ⟨3 * n + 1, by omega,
34         reaches_of_step_eq (collatzStep_eq_three_mul_add_one_of_not_even he)⟩
35
36 def rawBad : Set ℕ := {n | 0 < n ∧ ¬Reaches n 1}

```

```

37
38 theorem rawReaches_subset_bad {a : ℕ} (ha : ¬Reaches a 1) :
39   rawReaches a ⊆ rawBad := by
40   intro n hn
41   exact ⟨hn.1, fun h => ha (reaches_one_forward hn.2 h)⟩
42
43 theorem bad_three_unit_of_bad {n : ℕ} (hn : n ∈ rawBad) :
44   ∃ a : ℕ, a % 3 ≠ 0 ∧ ¬Reaches a 1 := by
45   obtain ⟨a, ha, hreach⟩ := reaches_three_unit n hn.1
46   exact ⟨a, ha, fun h => hn.2 (hreach.trans h)⟩
47
48 end CollatzTargetDensity

```

B.13 CollatzTargetDensity/CounterexampleDichotomy.lean

```

1 import CollatzTargetDensity.AllTargets
2 import CollatzTargetDensity.OrbitInvariance
3
4 /-!
5 # Consequence of the all-target density theorem
6
7 It does not establish that the sparse-counterexample criterion holds.
8 -/
9 namespace CollatzTargetDensity
10 open Erdos1135
11 noncomputable section
12
13 /-- One counterexample would force a positive lower natural density of
14   counterexamples. The density constant is not asserted to be uniform. -/
15 theorem counterexamples_positive_density (hbad : rawBad.Nonempty) :
16   ∃ c : ℝ, 0 < c ∧ ∃ X₀ : ℕ, ∀ X : ℕ, X₀ ≤ X →
17     c * (X : ℝ) ≤ (Terras.natCount rawBad X : ℝ) := by
18   obtain ⟨n, hn⟩ := hbad
19   obtain ⟨a, ha, hnot⟩ := bad_three_unit_of_bad hn
20   obtain ⟨c, hc, X₀, hX₀⟩ := positive_density_all_targets ha
21   refine ⟨c, hc, X₀, ?_⟩
22   intro X hX
23   have hmono := Terras.natCount_mono (rawReaches_subset_bad hnot) X
24   exact (hX₀ X hX).trans (by exact_mod_cast hmono)
25
26 /-- The counterexample proportion becomes arbitrarily small along arbitrarily
27   large cutoffs. This is a condition, not an established fact. -/
28 def SparseBadSubsequence : Prop :=
29   ∀ ε : ℝ, 0 < ε → ∀ X₀ : ℕ, ∃ X : ℕ, X₀ ≤ X ∧ 0 < X ∧
30     (Terras.natCount rawBad X : ℝ) < ε * (X : ℝ)
31
32 theorem collatz_iff_sparse_bad_subsequence :
33   (∀ n : ℕ, 0 < n → Reaches n 1) ↔ SparseBadSubsequence := by
34   constructor
35   · intro hall ε hε X₀
36     have hempty : rawBad = ∅ := by
37       ext n
38       constructor
39       · intro hn
40         exact False.elim (hn.2 (hall n hn.1))
41       · intro hn
42         exact False.elim hn
43     let X := max X₀ 1
44     have hXpos : 0 < X := by dsimp [X]; omega
45     have hcount : Terras.natCount rawBad X = 0 := by
46       rw [hempty]
47       simp [Terras.natCount]
48     refine ⟨X, le_max_left _, hXpos, ?_⟩
49     rw [hcount, Nat.cast_zero]
50     exact mul_pos hε (by exact_mod_cast hXpos)

```

```

51   • intro hsparse n hn
52     by_contra hnot
53     obtain ⟨c, hc, X0, hX0⟩ := counterexamples_positive_density ⟨n, hn, hnot⟩
54     obtain ⟨X, hX, _, hsmall⟩ := hsparse c hc X0
55     exact (not_lt_of_ge (hX0 X hX)) hsmall
56
57 end
58 end CollatzTargetDensity

```